

Database Systems

数据库系统

Instructor: Yiwen Shen (沈轶闻), Ph.D.
Dept. of Software & Computer Engineering,
Ajou University, Korea
chrisshen@ajou.ac.kr

今日概要

- 索引 (Index)

索引

大纲

- 基本概念
- 有序索引
- B⁺树索引文件
- B树索引文件
- 哈希
- 写入优化索引
- 时空索引

基本概念

- 用于加快访问所需数据的索引机制。
 - 例如，图书馆中的作者目录
- **搜索键** - 用于在文件中查找记录的属性集。
- **索引文件**由如下形式的记录（称为**索引条目**）组成：

搜索键	指针
-----	----

- 索引文件通常比原始文件小得多。
- 两种基本类型的索引：
 - 有序索引：搜索键按排序顺序存储
 - 哈希索引：使用“哈希函数”将搜索键均匀分布在“表”中。

索引评估指标

- 有效支持的访问类型。例如：
 - 属性中具有指定值的记录
 - 属性值落在指定值范围内的记录。
- 访问时间
- 插入时间
- 删除时间
- 空间开销

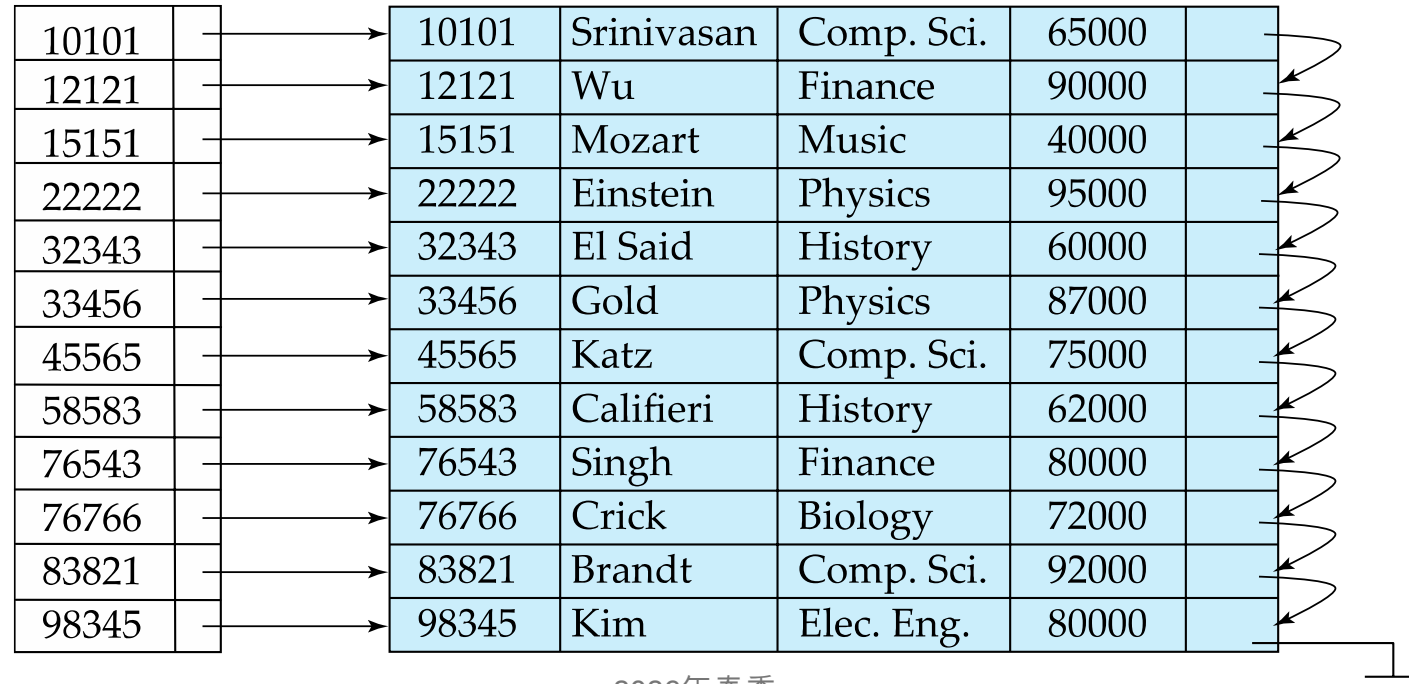
有序索引

- 在有序索引中，索引条目按搜索键值排序存储。
- 聚类索引（聚簇索引、主索引）：在顺序排列的文件中，索引的搜索键指定文件的顺序。
 - 主索引的搜索键通常是（但不一定是）主键。
- 非聚类索引（非聚簇索引、二级(次)索引、辅助索引）：索引的搜索键指定的顺序与文件的顺序不同。
- 索引顺序文件：按搜索键排序的顺序文件，搜索键上有聚类索引。

密集索引文件

- 密集索引 **Dense index** - 文件中每个搜索键值都会出现索引记录。
 - 例如: *instructor* 关系的ID属性的索引

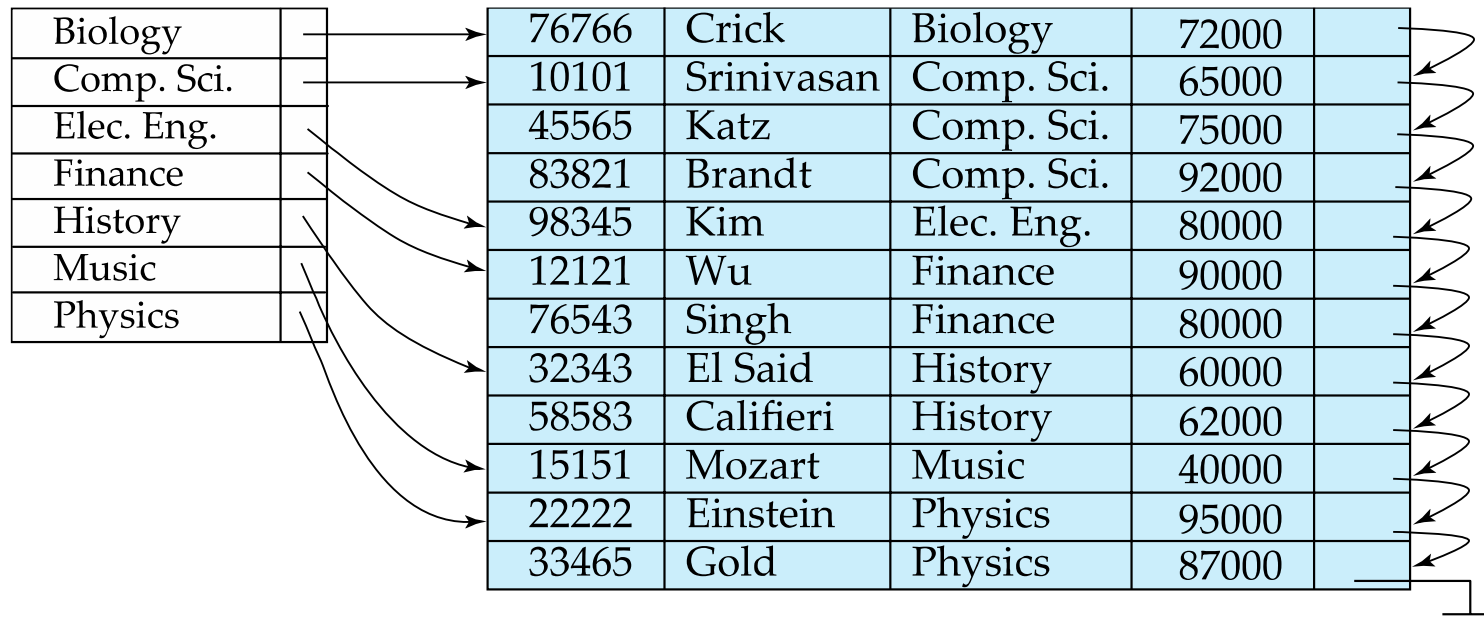
10101		→	10101	Srinivasan	Comp. Sci.	65000	
12121		→	12121	Wu	Finance	90000	
15151		→	15151	Mozart	Music	40000	
22222		→	22222	Einstein	Physics	95000	
32343		→	32343	El Said	History	60000	
33456		→	33456	Gold	Physics	87000	
45565		→	45565	Katz	Comp. Sci.	75000	
58583		→	58583	Califieri	History	62000	
76543		→	76543	Singh	Finance	80000	
76766		→	76766	Crick	Biology	72000	
83821		→	83821	Brandt	Comp. Sci.	92000	
98345		→	98345	Kim	Elec. Eng.	80000	



密集索引文件 (续)

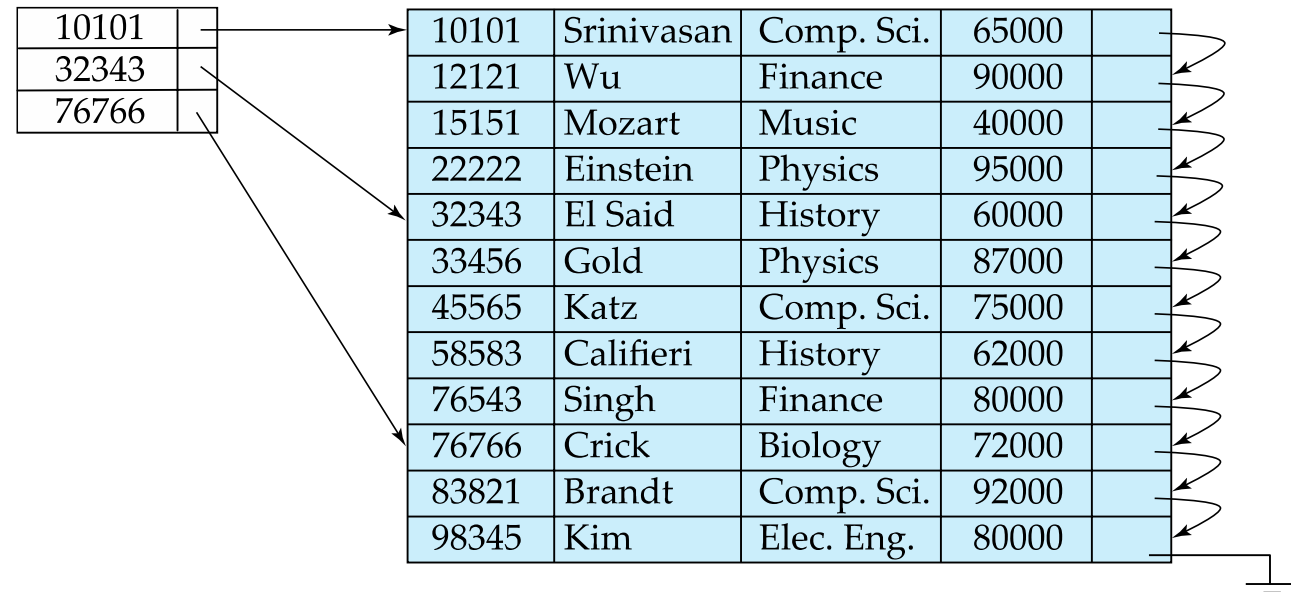
- 对 *dept_name* 建立密集索引, *instructor* 文件按 *dept_name* 排序。

Biology		76766	Crick	Biology	72000	
Comp. Sci.		10101	Srinivasan	Comp. Sci.	65000	
Elec. Eng.		45565	Katz	Comp. Sci.	75000	
Finance		83821	Brandt	Comp. Sci.	92000	
History		98345	Kim	Elec. Eng.	80000	
Music		12121	Wu	Finance	90000	
Physics		76543	Singh	Finance	80000	
		32343	El Said	History	60000	
		58583	Califieri	History	62000	
		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		33465	Gold	Physics	87000	



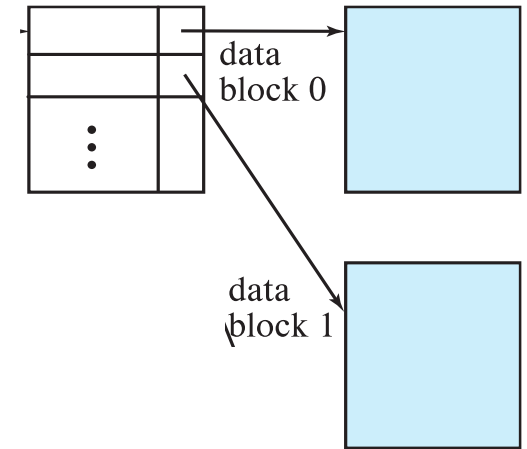
稀疏索引文件

- **稀疏索引**：仅包含部分搜索键值的索引记录。
 - 适用于记录按搜索键顺序排列的情况
- 要查找搜索键值为 K 的记录，我们：
 - 查找搜索键值大于 K 的最大索引记录
 - 从索引记录指向的记录开始，按顺序搜索文件。

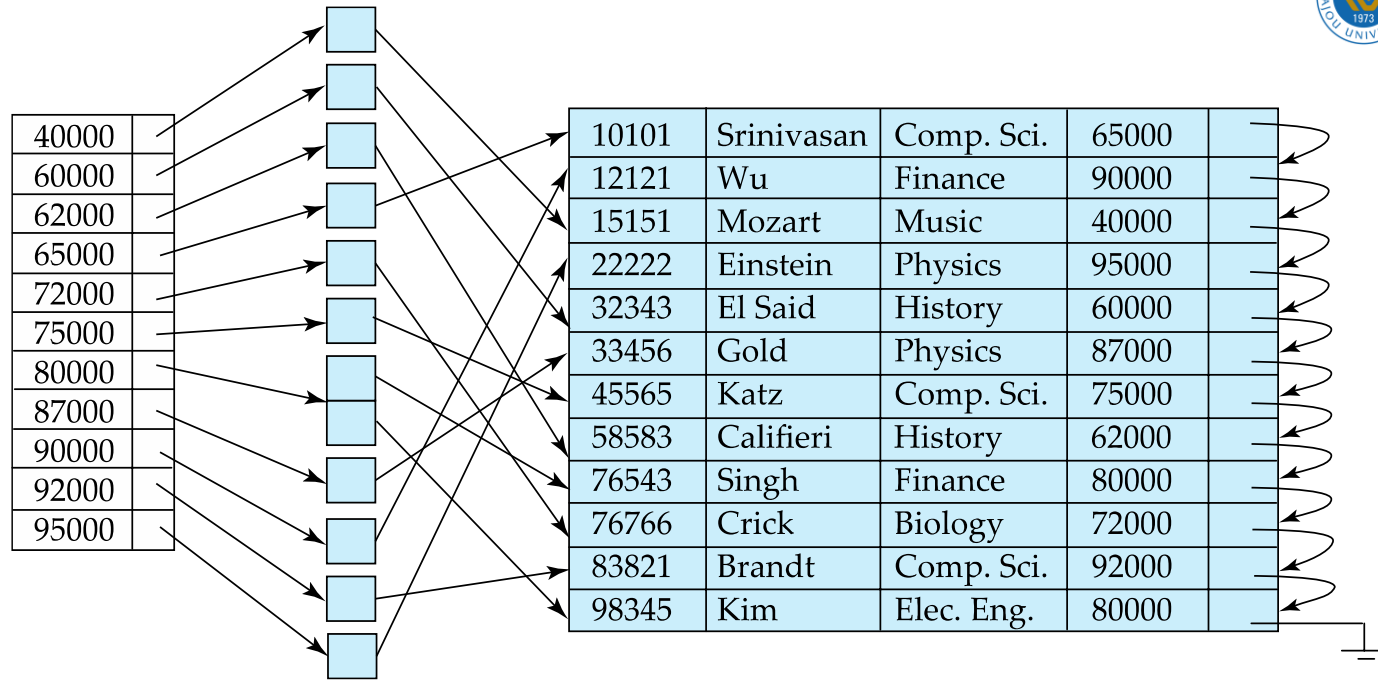


稀疏索引文件 (续)

- 与密集索引相比：
 - 插入和删除操作占用空间更小，维护成本也更低。
 - 通常比密集索引查找记录的速度慢。
- 好的权衡：
 - 对于聚类索引：稀疏索引，其中每个文件块都有一个索引条目，对应于块中最小的搜索键值。
 - 对于非聚类索引：在密集索引之上构建稀疏索引（多级索引）



次索引示例

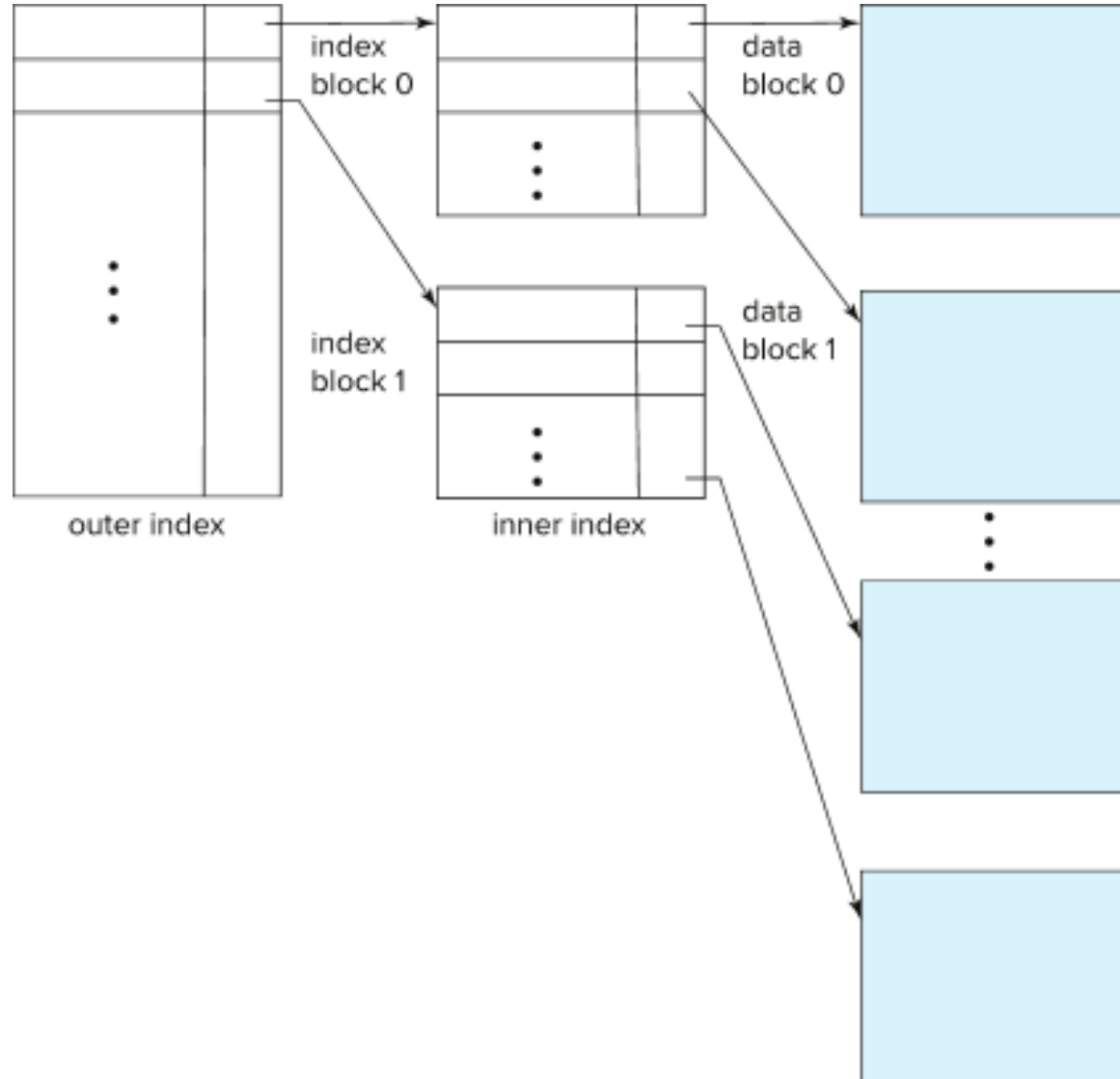


- instructor 薪资领域的二级索引 (次索引)
- 索引记录指向一个存储表 (bucket), 该存储表包含指向所有具有该特定搜索键值的实际记录的指针。
- 二级指数必须足够密集。

多级索引

- 如果索引无法放入内存，访问成本就会很高。
- 解决方案：将保存在磁盘上的索引视为顺序文件，并对其构建稀疏索引。
 - 外部索引 - 基本索引的稀疏索引
 - 内部索引 - 基本索引文件
- 如果连最外层索引都太大而无法放入主内存，则可以创建另一级索引，依此类推。
- 当文件中插入或删除数据时，所有级别的索引都必须更新。

多级索引 (续)



多键索引

- 复合搜索键

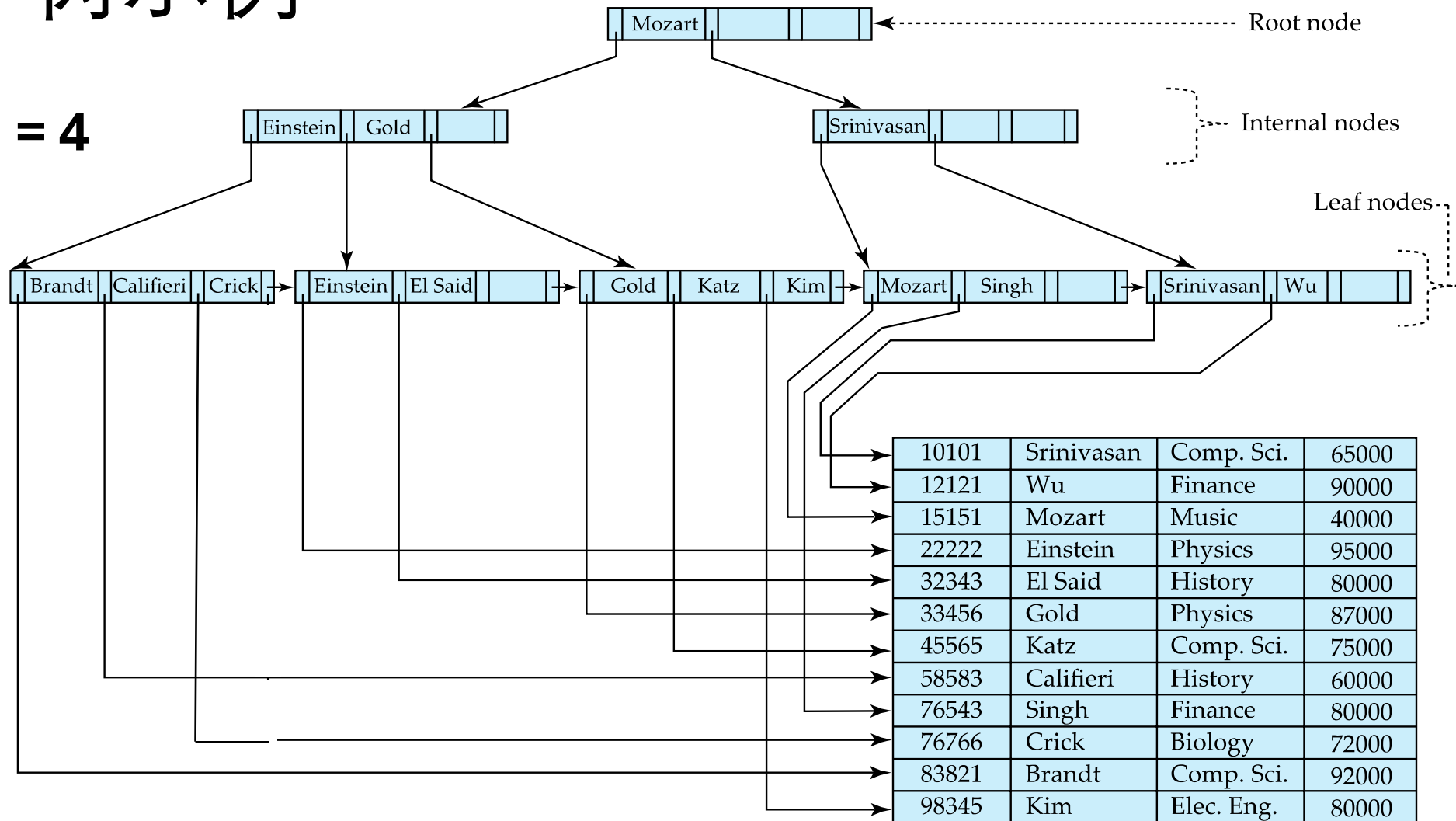
- 例如：在 *instructor* 关系的属性 (*name*, *ID*) 上建立索引
- 值按字典顺序排序
 - 例如：(John, 12121) < (John, 13514) 且
(John, 13514) < (Peter, 11223)
- 可以只按 *name* 查询，也可以按 (*name*, *ID*) 查询。

B⁺树索引文件

- 索引顺序文件的缺点
 - 随着文件增大，性能会下降，因为会创建许多溢出块。
 - 需要定期对整个文件进行重新整理。
- B⁺树索引文件的优点：
 - 面对插入和删除操作，它会自动进行局部小的调整，从而实现自身的重组。
 - 无需对整个文件进行重新组织即可保持性能。
- B⁺树的（小）缺点：
 - 额外的插入和删除开销，空间开销。
- B⁺树的优点大于缺点
 - 所以它被广泛使用

B⁺树示例

n = 4



B⁺树索引文件 (续)

一个B⁺树是一棵有根树, 满足以下性质:

- 从根到叶的所有路径长度都相同。
- 每个非根节点或非叶节点 (也就是中间/内部节点) 都有 $\lceil n/2 \rceil$ 到 n 个子节点。
- 叶节点的搜索键值介于 $\lceil (n-1)/2 \rceil$ 和 $n-1$ 之间
- 特殊情况:
 - 如果根节点不是叶子, 则它至少有 2 个子节点。
 - 如果根节点是叶子节点 (即树中没有其他节点), 则其值介于 0 和 $(n-1)$ 之间。

B⁺-树节点结构

- 典型节点

P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-----	-----------	-----------	-------

- K_i 是搜索键值 (search-key values)
- P_i 是指向子节点的指针 (对于非叶子节点) 或指向记录或记录表的指针 (对于叶子节点)。
- 节点中的搜索键是有序的。

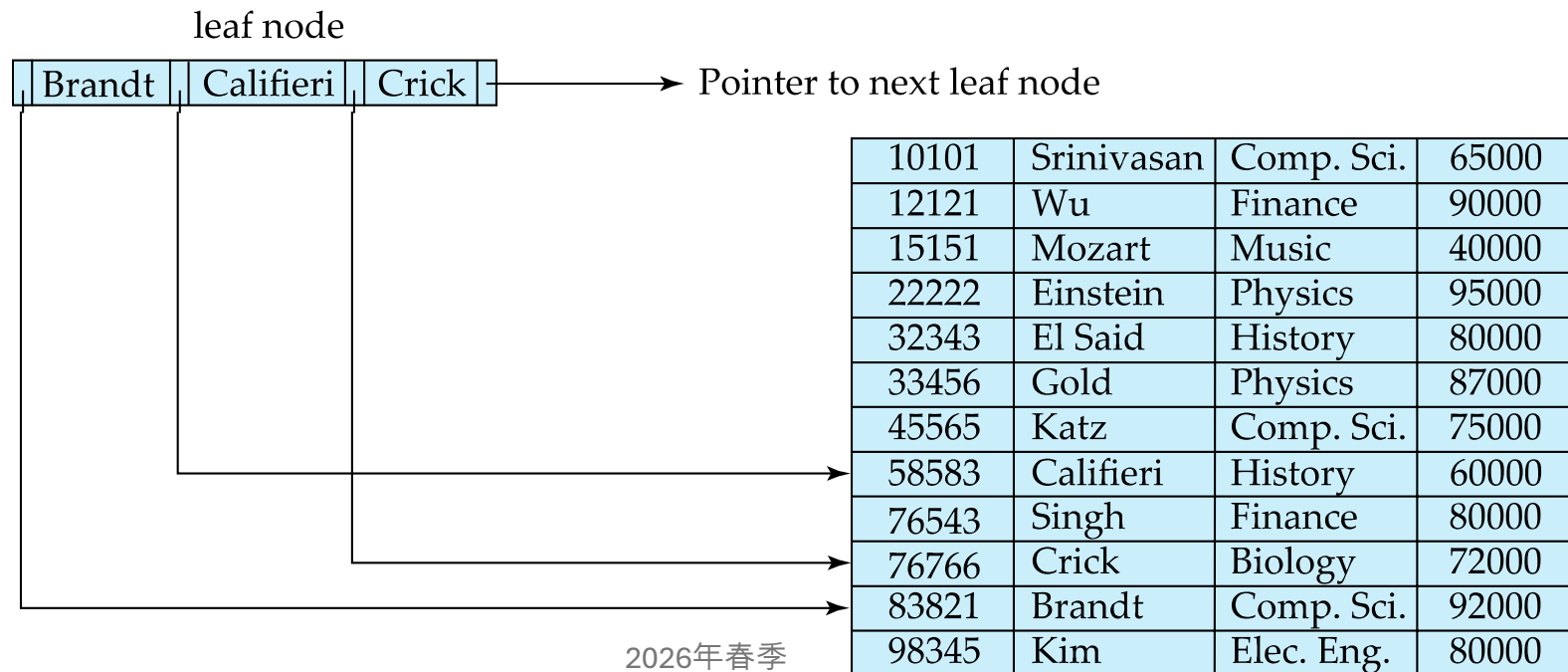
$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$

(最初假设没有重复键, 重复键的问题稍后再处理)

B⁺ 树中的叶节点

叶节点的属性:

- 对于 $i = 1, 2, \dots, n-1$, 指针 P_i 指向搜索键值为 K_i 的文件记录,
- 如果 L_i, L_j 是叶节点, 且 $i < j$, L_i 的搜索键值小于或等于 L_j 的搜索键值
- P_n 指向搜索键顺序中的下一个叶节点



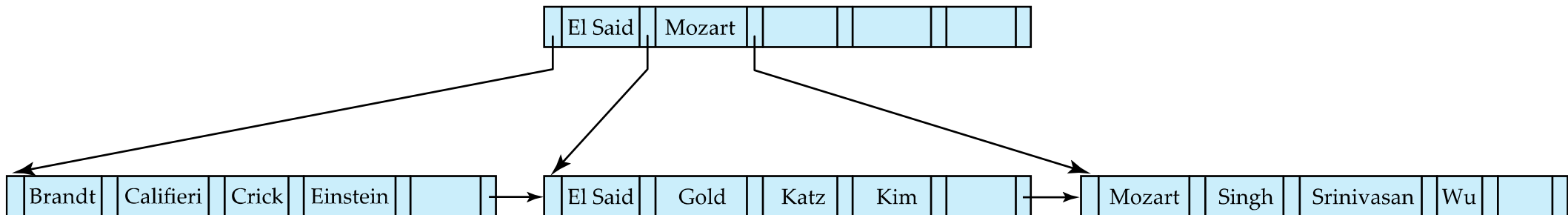
B⁺ 树中的非叶节点

- 非叶节点在叶节点上形成多级稀疏索引。对于具有 m 个指针的非叶节点：
 - 所有 P_i 指向的子树中搜索键是小于 K_i 的
 - 对于 $2 \leq i \leq n-1$, P_i 指向的子树中所有搜索键的值大于或等于 K_{i-1} 且小于 K_i 的值。
 - P_n 指向的搜索键的值大于或等于 K_{n-1} 的键
 - 总体结构

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

B⁺树示例

- *instructor* 的B⁺树文件 ($n=6$)

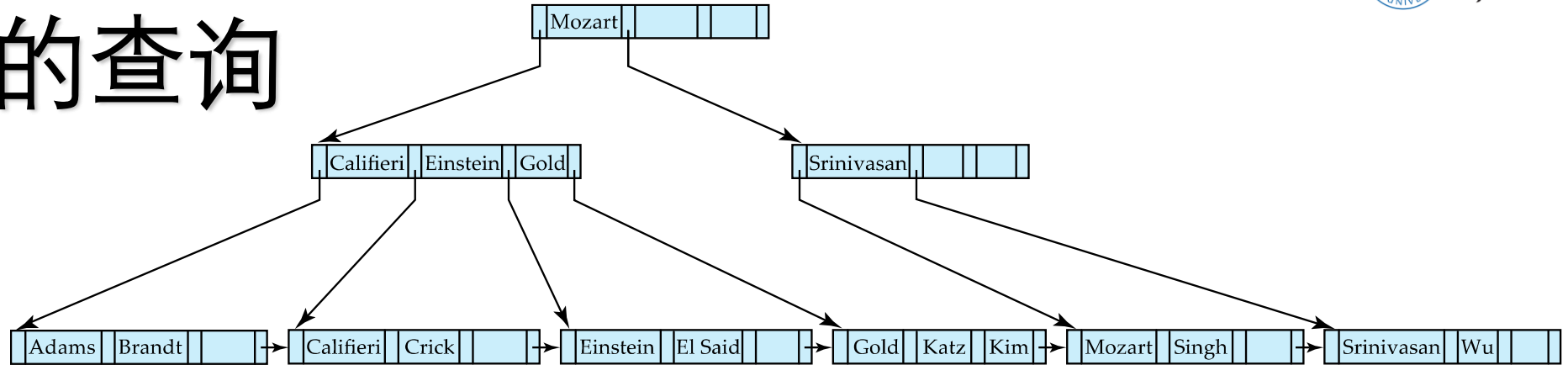


- 叶节点必须具有 3 到 5 个值 ($\lceil (n-1)/2 \rceil$ 和 $n-1$, 其中 $n=6$) 。
- 除根节点外的非叶节点必须有 3 到 6 个子节点 ($\lceil n/2 \rceil$ 且 $n=6$) 。
- Root 必须至少有 2 个子节点。

B⁺树的观察

- 由于节点间的连接是通过指针实现的，“逻辑上”关闭的块不一定“物理上”关闭。
- B⁺树的非叶子层构成稀疏索引的层次结构。
- B⁺树包含的层数相对较少。
 - 根以下层至少有 $2 * \lceil n/2 \rceil$ 值 (搜索键值)
 - 下一级别至少有 $2 * \lceil n/2 \rceil * \lceil n/2 \rceil$ 值 (搜索键值)
 - .. 等等。
 - 如果文件中存在 K 个搜索键值，则树的高度不超过 $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
 - 因此，可以高效地进行搜索。
- 对主文件的插入和删除操作可以高效地处理，因为索引可以在对数时间内重构（之后我们将会看到）。

B⁺树上的查询

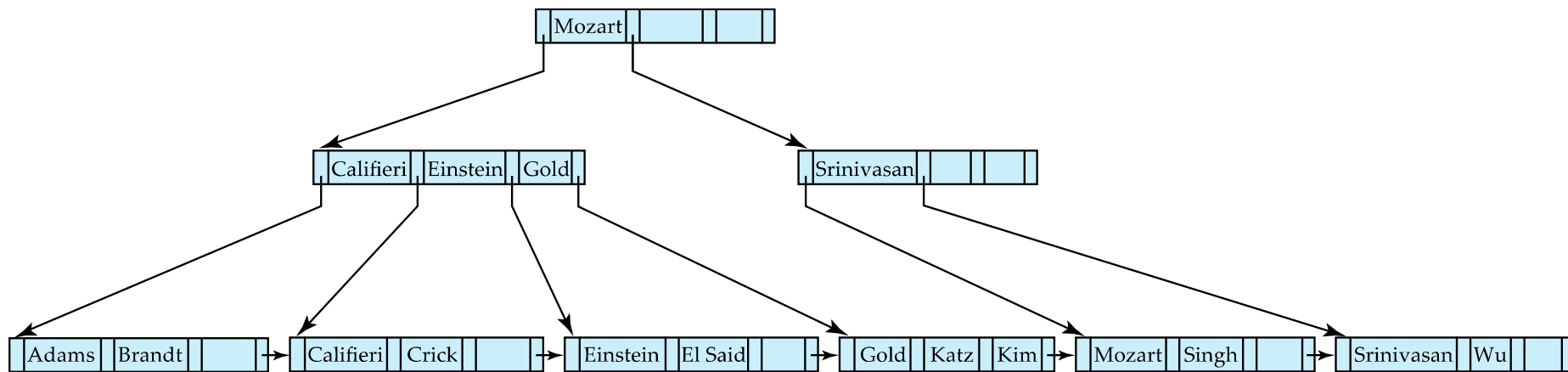


function find(v)

1. $C = \text{root}$
2. while (C is not a leaf node)
 1. Let i be least number s.t. $v \leq K_i$.
 2. if there is no such number i then
 3. Set $C = \text{last non-null pointer in } C$ 在当前节点的最后一个搜索键
 4. else if ($v = C.K_i$) Set $C = P_{i+1}$ 找到该搜索键
 5. else set $C = C.P_i$ 上述两个情况都不成立，移到下一层搜索
3. if for some i , $K_i = v$ then return $C.P_i$
4. else return null /* no record with search-key value v exists. */

B⁺树上的查询 (续)

- **范围查询**查找给定范围内所有具有搜索键值的记录
 - 函数 *findRange* (*lb*, *ub*) 返回所有此类记录的集合
 - 实际实现通常提供一个迭代器接口，使用 `next()` 函数一次获取一条匹配的记录。



B⁺-树的查询 (续)

- 如果文件中有 K 个搜索键值，则树的高度不超过 $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$ 。
- 一个节点的大小通常与一个磁盘数据块的大小相同，通常为 4 kilobytes。
 - n 通常约为 100 (每个索引条目 40 字节)。 $\rightarrow 4000/40 = 100$
- 包含 100 万个搜索键值， $n = 100$
 - 从根到叶的查找遍历最多访问 $\log_{50}(1,000,000) = 4$ 个节点。
- 与之形成对比的是，一个拥有 100 万个搜索键值的平衡二叉树 - 每次查找操作大约只会访问 20 个节点。
 - 上述差异意义重大，因为每次节点访问都可能需要磁盘 I/O，耗时约 20 毫秒。

Result

Answer: $y = 3.5315514604067$ $\log_{50}(1000000) = 3.5315514604067$
 $50^{3.5315514604067} = 1000000$

log	1000000	=	
	50		
Calculate		Clear	

非唯一键

- 如果搜索键 a_i 不唯一，则应基于复合键 (a_i, A_p) 创建索引，该复合键是唯一的。
 - A_p 可以是主键、记录 ID 或任何其他能保证唯一性的属性。
- 可以通过对复合键进行范围搜索来查找 $i = v$ ，范围为 $(v, -\infty)$ 到 $(v, +\infty)$ 。
- 但还需要更多 I/O 操作才能获取实际记录。
 - 如果索引是聚类索引，则所有访问都是顺序的。
 - 如果索引是非聚类索引，则每次记录访问都可能都需要一次 I/O 操作。

B⁺树的更新：插入操作

假设记录已添加到文件 (数据库) 中。

- pr 是指向记录的指针，并且让
- v 是记录的搜索键值

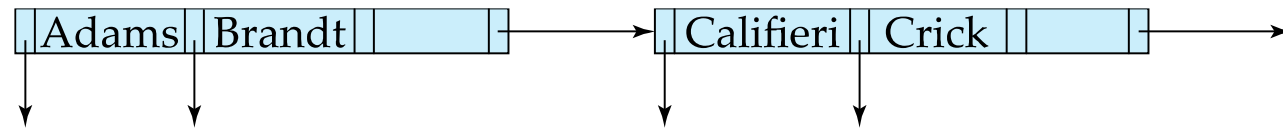
1. 找到包含搜索键值的叶节点。

1. 如果叶节点中有空间，则在叶节点中插入 (v, pr) 对。
2. 否则，按照下一张幻灯片中讨论的方式拆分节点（连同新的 (v, pr) 条目），并将更新传播到父节点。

B⁺树的更新：插入操作（续）

• 拆分叶节点：

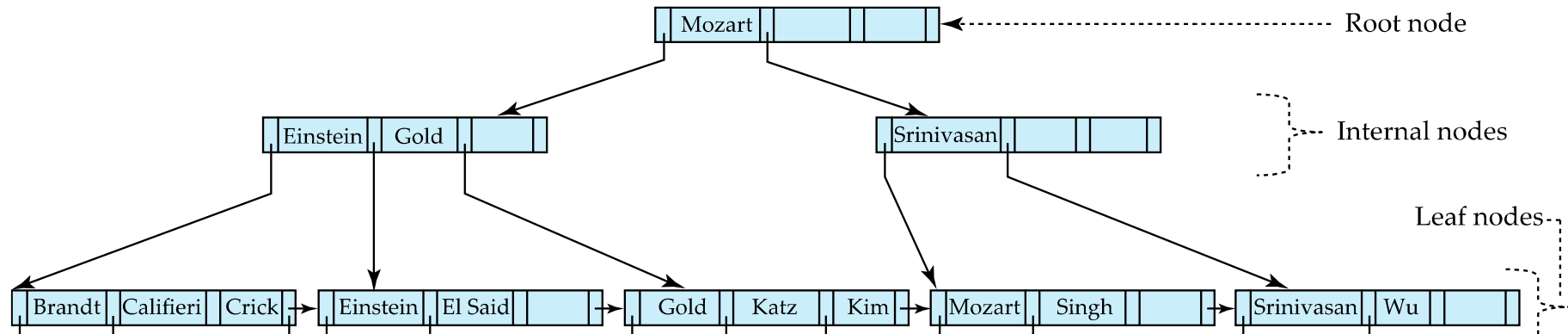
- 取出 n 个（搜索键值，指针）对（包括要插入的那个）。将第一个 $\lceil n/2 \rceil$ 放入原节点，其余的放入新节点。
 - 设新节点为 p ，设 k 为 p 中最小的键值。将 (k, p) 插入到被拆分节点的父节点中。
 - 如果父级已满，则将其拆分，并将拆分结果向上级传播。
- 节点拆分向上进行，直到找到一个未满的节点为止。
- 最坏的情况下，根节点可能会分裂，导致树的高度增加 1。



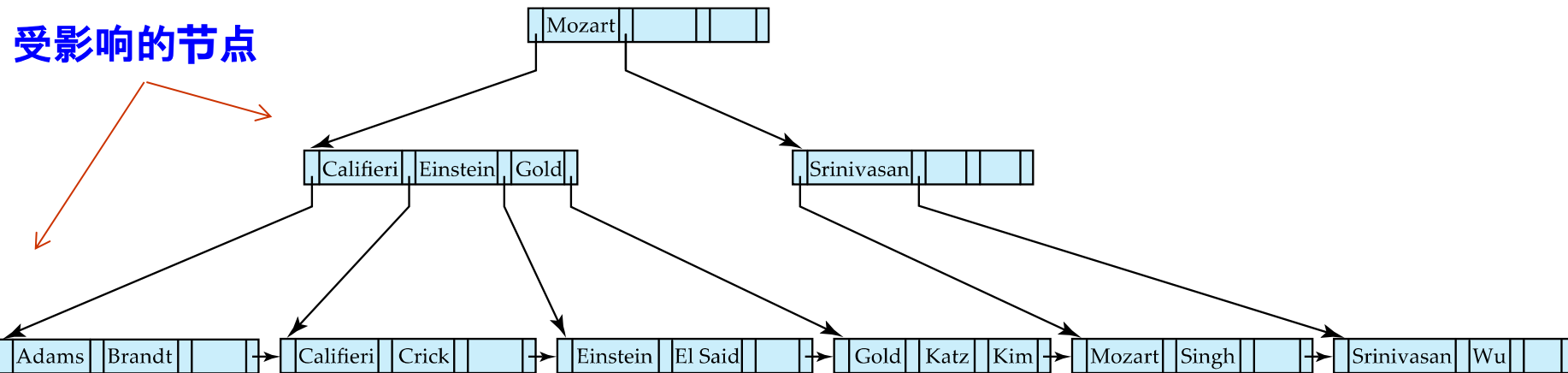
Califieri和 Crick 的节点在插入 Adams 后分裂的结果

下一步：在父节点中插入包含 (Califieri , 指向新节点的指针)的条目

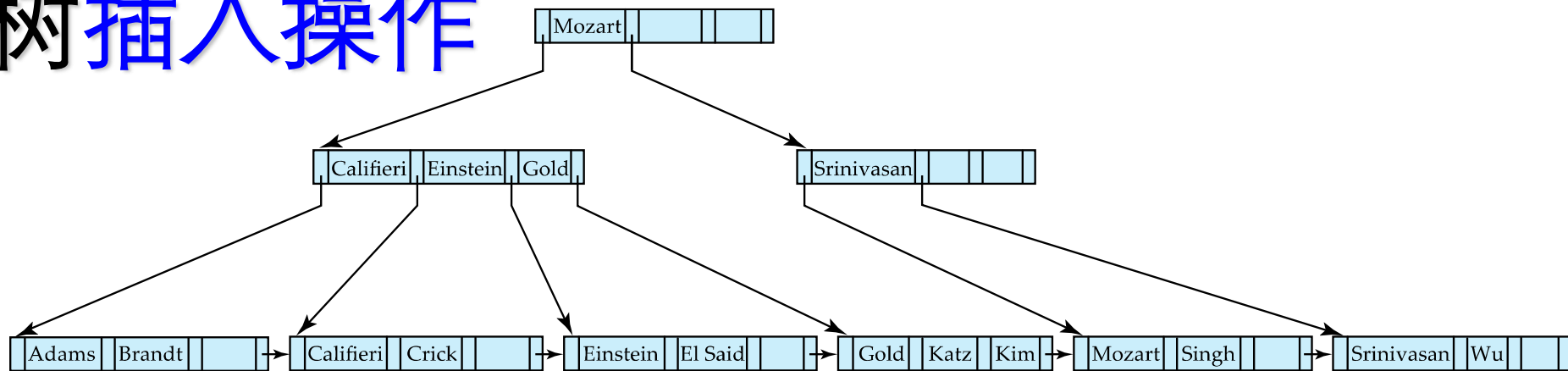
B⁺-树插入操作



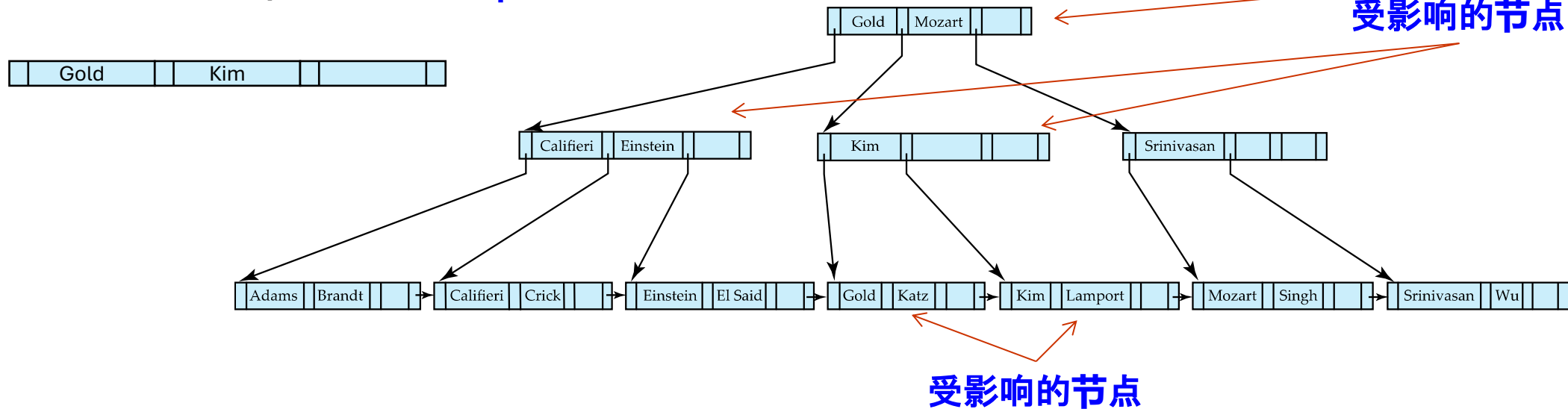
在插入“Adams”之前和之后



B⁺-树插入操作

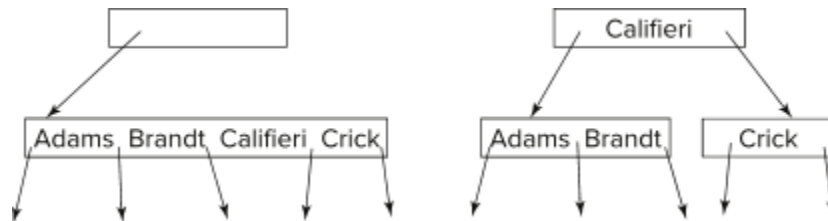


B⁺树在插入“Lampport”之前和之后

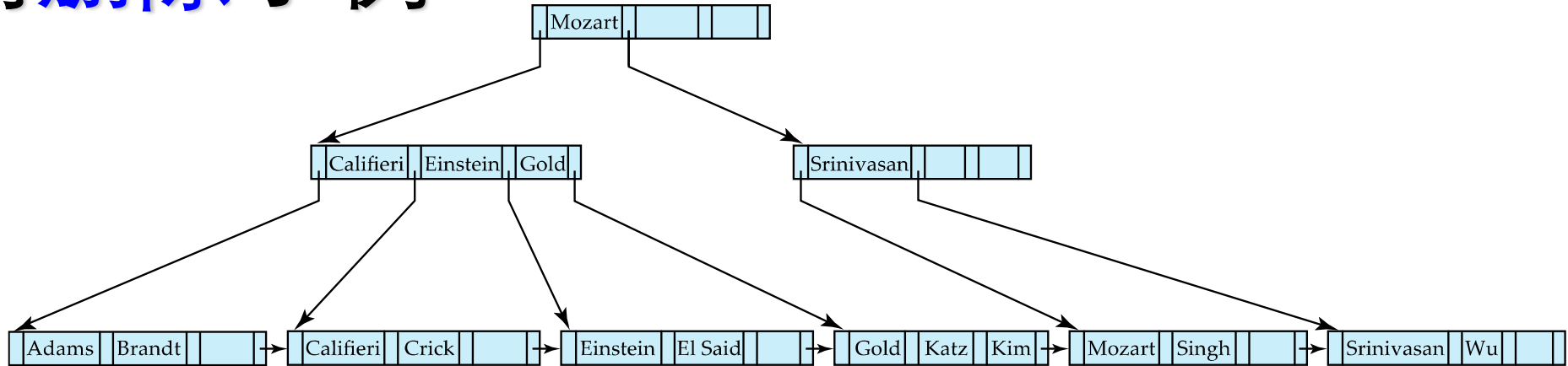


B⁺树中的插入操作 (续)

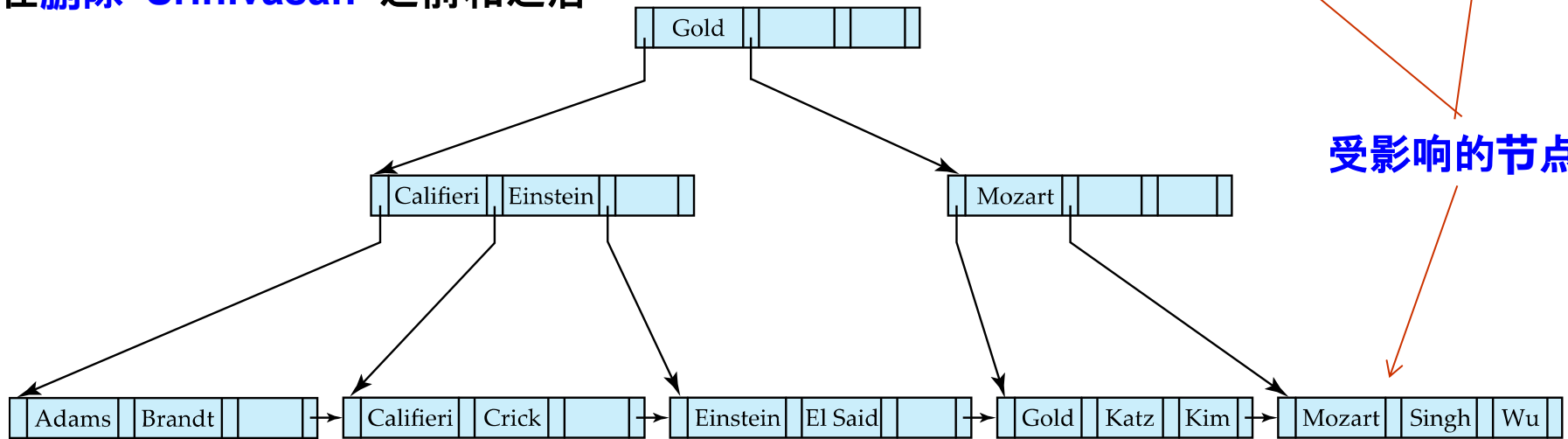
- 拆分**非叶节点**：当将 (k, p) 插入到已满的内部节点 N 时
 - 将 N 复制到内存区域 M 中，该区域可容纳 $n+1$ 个指针和 n 个键。
 - 将 (k, p) 插入 M
 - 将节点 M 中的 $P_1, K_1, \dots, K_{\lceil n/2 \rceil - 1}, P_{\lceil n/2 \rceil}$ 复制回节点 N
 - 从节点 M 中复制 $P_{\lceil n/2 \rceil + 1}, K_{\lceil n/2 \rceil + 1}, \dots, K_n, P_{n+1}$ 放入新分配的节点 N'
 - 插入 $(K_{\lceil n/2 \rceil}, N')$ 进入父级 N
- 例子



B⁺树删除示例



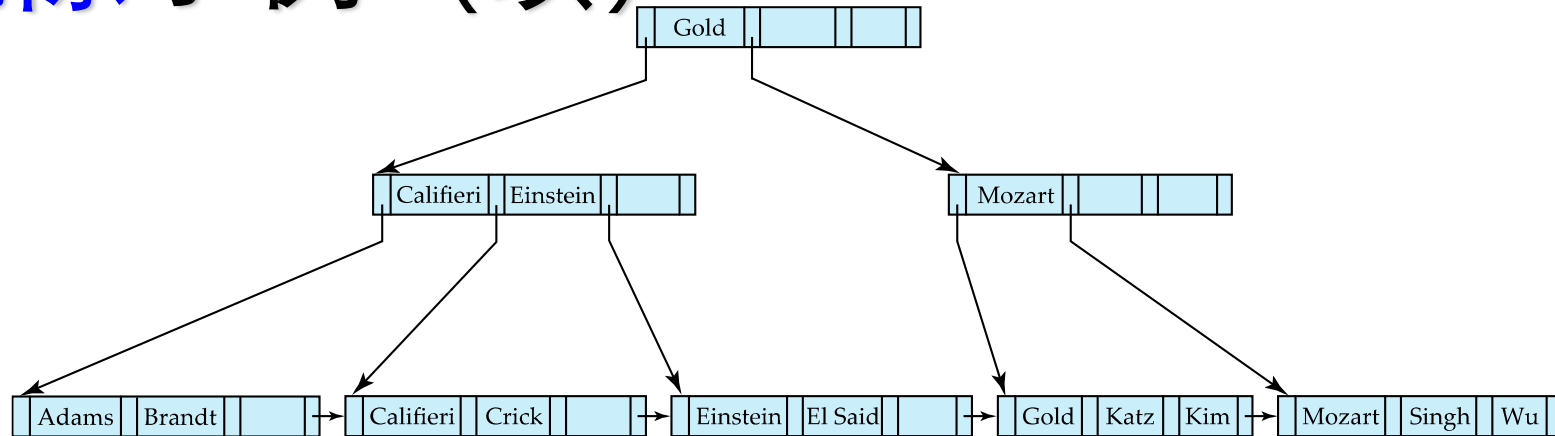
在删除“Srinivasan”之前和之后



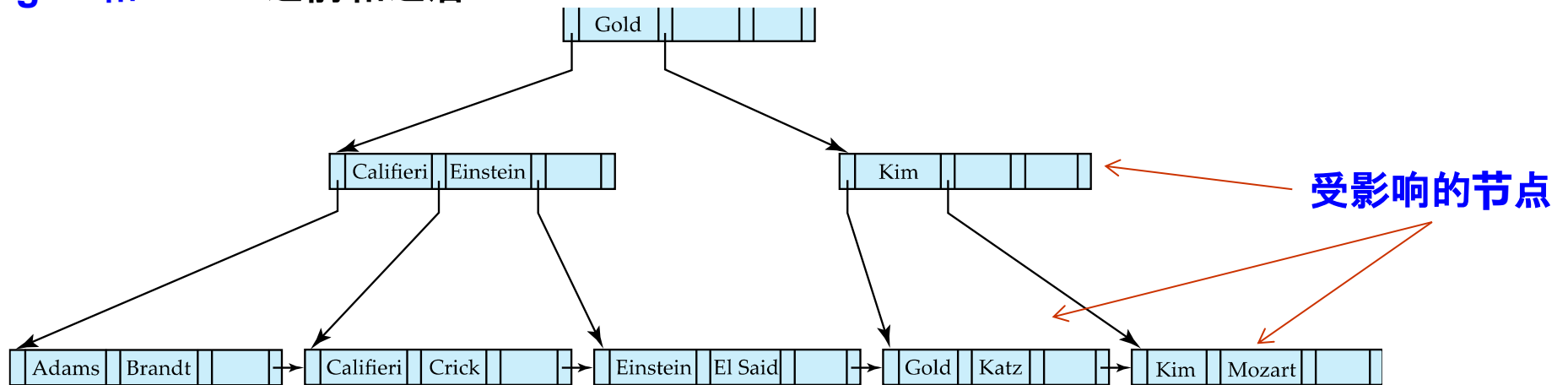
受影响的节点

删除“Srinivasan”会导致合并叶片

B⁺树删除示例 (续)

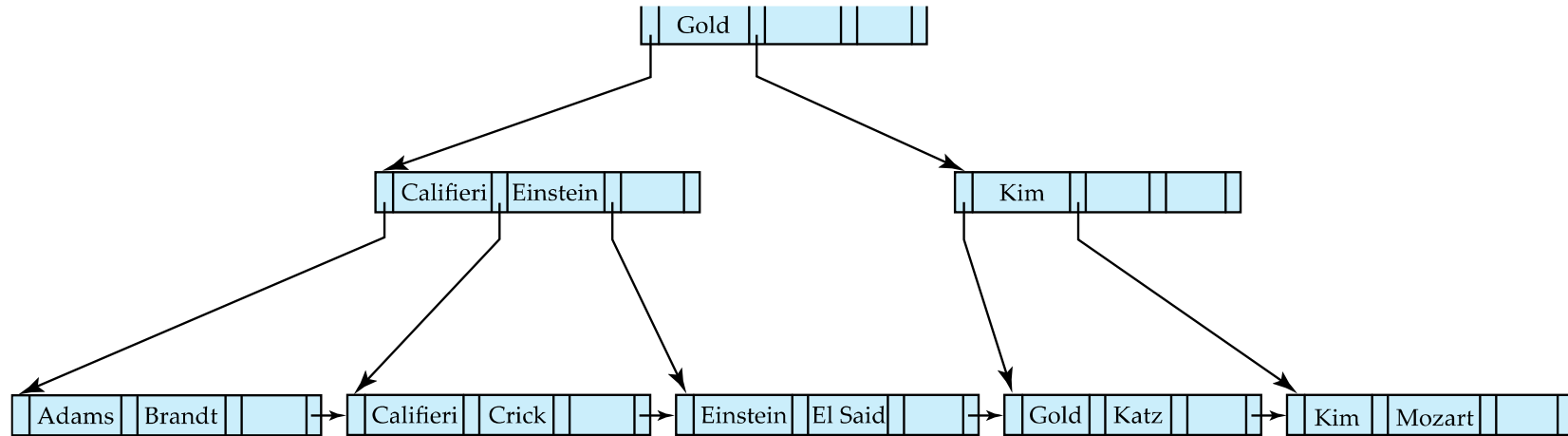


在删除“Singh”和“Wu”之前和之后

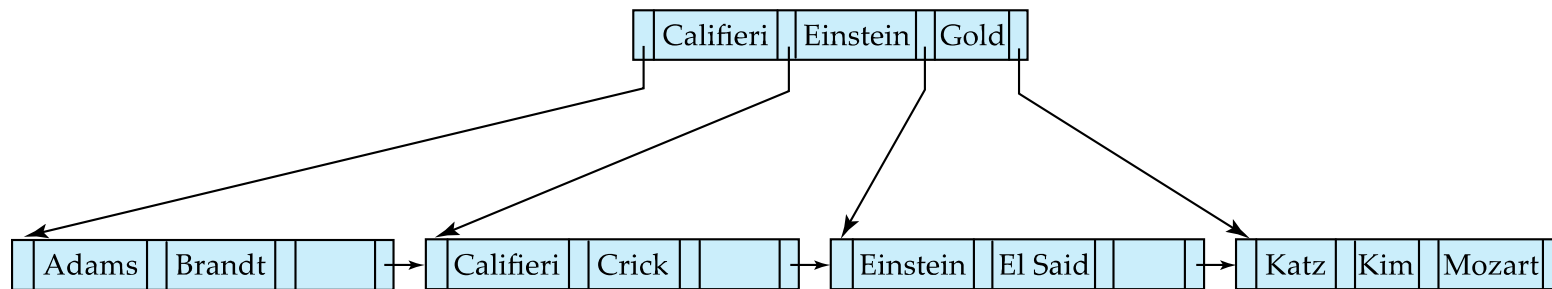


包含 Singh 和 Wu 的叶子变得不满，并从其左侧兄弟叶借用了 Kim 值。
父级中的搜索键值因此发生变化

B⁺树删除示例 (续)



在删除“Gold”之前和之后：



- 包含 **Gold** 和 **Katz** 的节点容量不足，因此与其兄弟节点合并。
- 父节点容量不足，与其兄弟节点合并。
 - 合并时，分隔两个节点（在父节点处）的值会被下拉。
- 根节点此时只剩下一个子节点，并被删除。

B⁺树的更新: 删除

假设记录已从文件中删除。令 V 为该记录的搜索键值, P_r 成为指向记录的指针。

- 从叶节点中移除 (P_r, V)
- 如果 删除操作导致节点条目过少, 并且该节点及其同级节点中的条目数量足以合并到一个节点中, 则进行合并兄弟节点:
 - 将两个节点中的所有搜索键值插入到一个节点 (左侧的节点) 中, 并删除另一个节点。
 - 使用上述过程, 从其父节点递归删除 (K_{i-1}, P_i) 对, 其中 P_i 是指向被删除节点的指针。

B⁺树的更新: 删除

- 否则, 如果由于删除操作导致节点**条目过少**, 但该节点及其同级节点中的条目**不足以放入单个节点**, 则**重新分配指针**:
 - 重新分配节点与其兄弟节点之间的指针, 使得两者都具有**大于最小条目数的条目**。
 - 更新节点父节点中对应的搜索键值。
- 找到具有 $\lceil n/2 \rceil$ 或更多指针的节点为止。
- 如果删除后根节点**只有一个指针**, 则该根节点将被删除, 其唯一的子节点将成为新的根节点。
- 注意: 值得注意的是, 由于删除操作, B⁺ 树的非叶子节点中存在的键值可能在树的任何叶子节点中都不存在。

更新的复杂性

- 插入和删除单个条目的成本（以 I/O 操作次数衡量）与**树的高度成正比**。
 - 对于 K 个条目和最大扇出为 n 的情况，插入/删除一个条目的**最坏情况复杂度为 $O(\log_{\lceil n/2 \rceil}(K))$**
- 实际上，I/O 操作次数更少：
 - 内部节点通常位于**缓冲区**中。
 - **拆分/合并操作很少见**，大多数插入/删除操作只会影响**叶节点**。
- 平均节点占用率取决于插入顺序
 - 三分之二随机插入，一半按排序顺序插入。

非唯一搜索键值

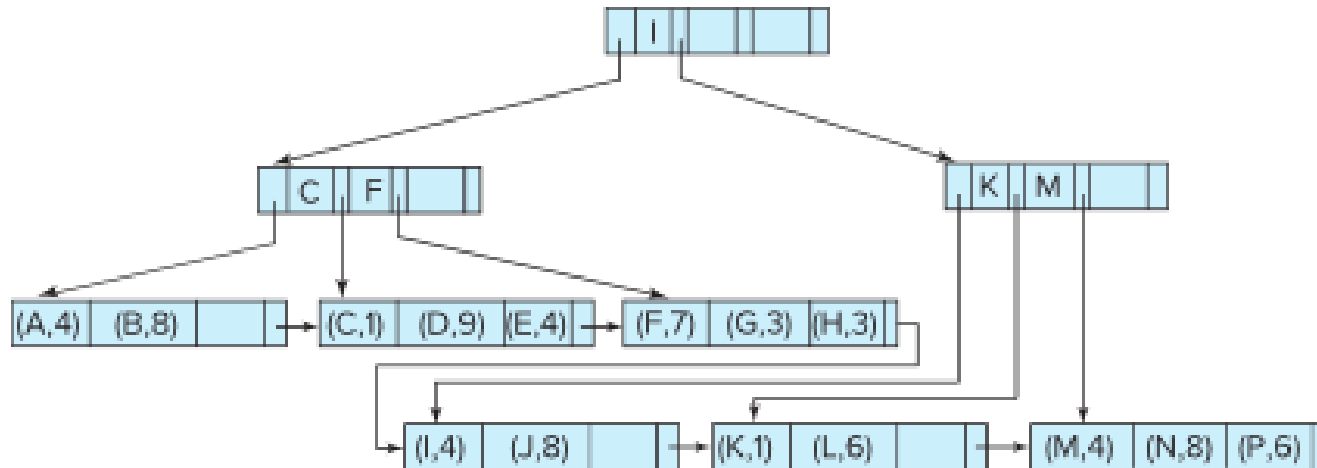
- 前述方案 (复合搜索键) 的替代方案
 - 单独的存储表放在单独的区块上 (不好的做法)
 - 包含每个键的元组指针列表
 - 需要额外代码处理长列表
 - 如果搜索键存在大量重复项, 则删除元组的开销可能很大 (为什么?)
 - 最坏情况下的复杂度可能是线性的!
 - 空间占用低, 查询无额外费用
 - 通过添加记录标识符使搜索键唯一
 - 额外的键存储空间
 - 更简洁的插入/删除代码
 - 广泛使用

B⁺树文件组织

- B⁺树文件组织 (File Organization):
 - B⁺树文件组织中的叶子节点存储的是记录(数据)，而不是指针。
 - 即使发生插入/删除/更新操作，也能帮助保持数据记录的聚类。
- 叶节点仍然需要半满。
 - 由于记录比指针大，因此叶节点中可存储的最大记录数小于非叶节点中的指针数。
- B⁺树索引中条目的插入和删除操作的处理方式相同。

B⁺树文件组织 (续)

- B⁺树文件组织示例



- 好的空间利用率非常重要，因为记录比指针占用更多空间。
- 为了提高空间利用率，在分裂和合并过程中，应让更多兄弟节点参与重新分配。
 - 让两个同级节点参与重新分配（尽可能避免分裂/合并）可使每个节点至少拥有 $\lfloor 2n/3 \rfloor$ 条目

索引中的其他问题

- 记录迁移 和 二级索引

- 如果记录发生移动，所有存储记录指针的二级索引都必须更新。
- 在 B⁺树文件组织中，节点分裂的成本会变得非常高。
- 解决方案：在二级索引中使用 B⁺树文件组织的搜索键，而不是记录指针。
 - B⁺树文件组织搜索键不唯一，则添加记录 ID
 - 额外遍历文件组织结构以查找记录
 - 查询成本较高，但节点拆分成本较低。

索引字符串

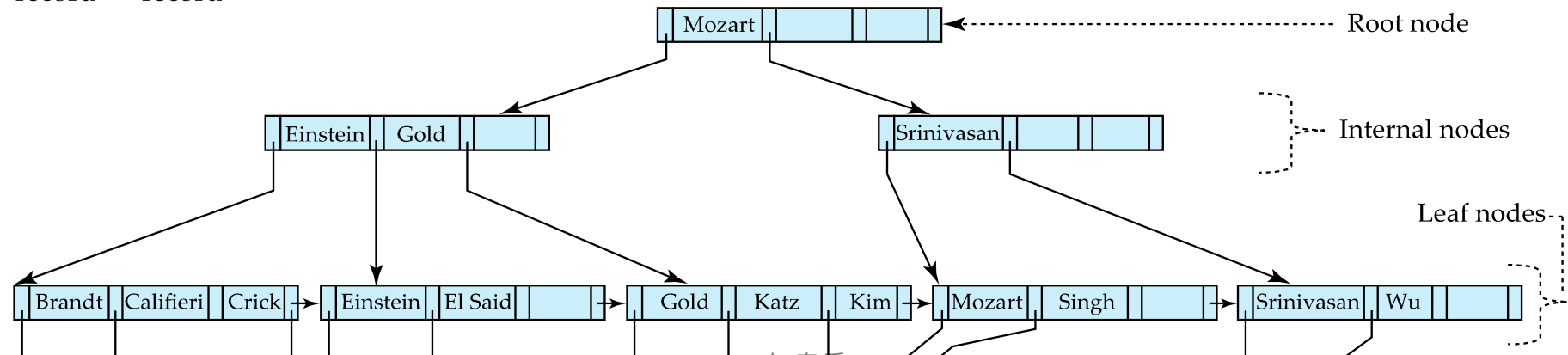
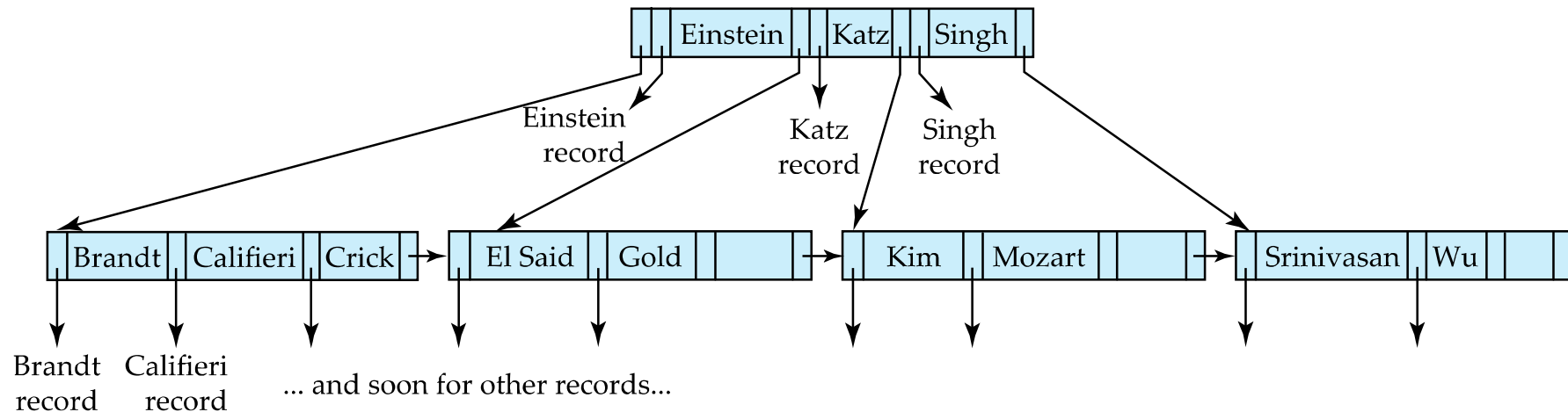
- 可变长度字符串作为键
 - 可变扇出
 - 使用空间利用率作为拆分标准，而不是指针数量。
- 前缀压缩
 - 内部节点的键值可以是完整键的前缀
 - 保留足够的字符以区分由键值分隔的子树中的条目
 - 例如，“Silas”和“Silberschatz”可以用“Silb”
 - 叶节点中的键可以通过共享公共前缀进行压缩。

批量装载和自下而上构建

- 将条目逐个插入到 B^+ 树中，每个条目至少需要1次 I/O 操作。
 - 假设叶子层级的数据无法全部放入内存
 - 一次性加载大量条目（批量加载）效率可能非常低下。
- 高效方案一：
 - 首先对条目进行排序
 - 按排序顺序插入
 - 插入内容将插入到现有页面（或导致页面拆分）
 - IO性能大幅提升，但大多数叶节点仍处于半满状态。
- 高效方案二：自下而上的 B^+ 树构建
 - 与之前一样，对条目进行排序
 - 然后从叶级开始，逐层创建树状结构。
 - 大多数数据库系统都将其作为批量加载实用程序的一部分实现。

B树索引文件示例

基于相同数据的 B 树（上图）和 B+ 树（下图）



闪存索引

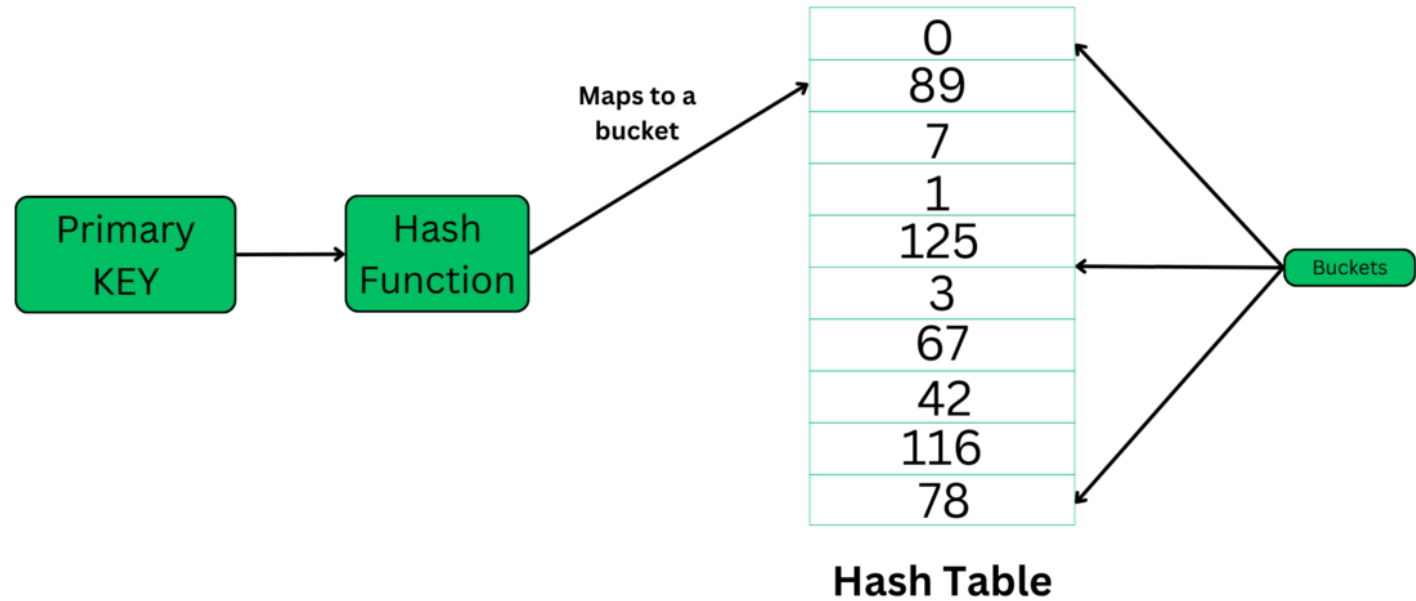
- 闪存的随机 I/O 成本要低得多。
 - 读/写时间为 20 至 100 微秒
- 写入操作并非原地执行，最终需要更昂贵的擦除操作。
- 因此，最佳页面尺寸要小得多。
- 批量加载仍然很有用，因为它能最大限度地减少页面擦除。
- 针对闪存优化的搜索树，我们采用了写入优化的树结构（稍后讨论）来最大限度地减少页面写入。

主内存中的索引

- 内存随机访问
 - 比在磁盘/闪存上便宜得多。
 - 但与缓存读取相比仍然很昂贵
 - 优先选择能够最大限度利用缓存的数据结构。
 - B⁺树节点中对键值进行二分查找会导致多次缓存未命中。
- 有小节点的B⁺树 (能够放入缓存行) 是更可取的选择从而减少缓存未命中。
- 核心思想：使用较大的节点尺寸来优化磁盘访问，但使用较小的节点尺寸的树来组织节点内的数据，而不是使用数组。

哈希

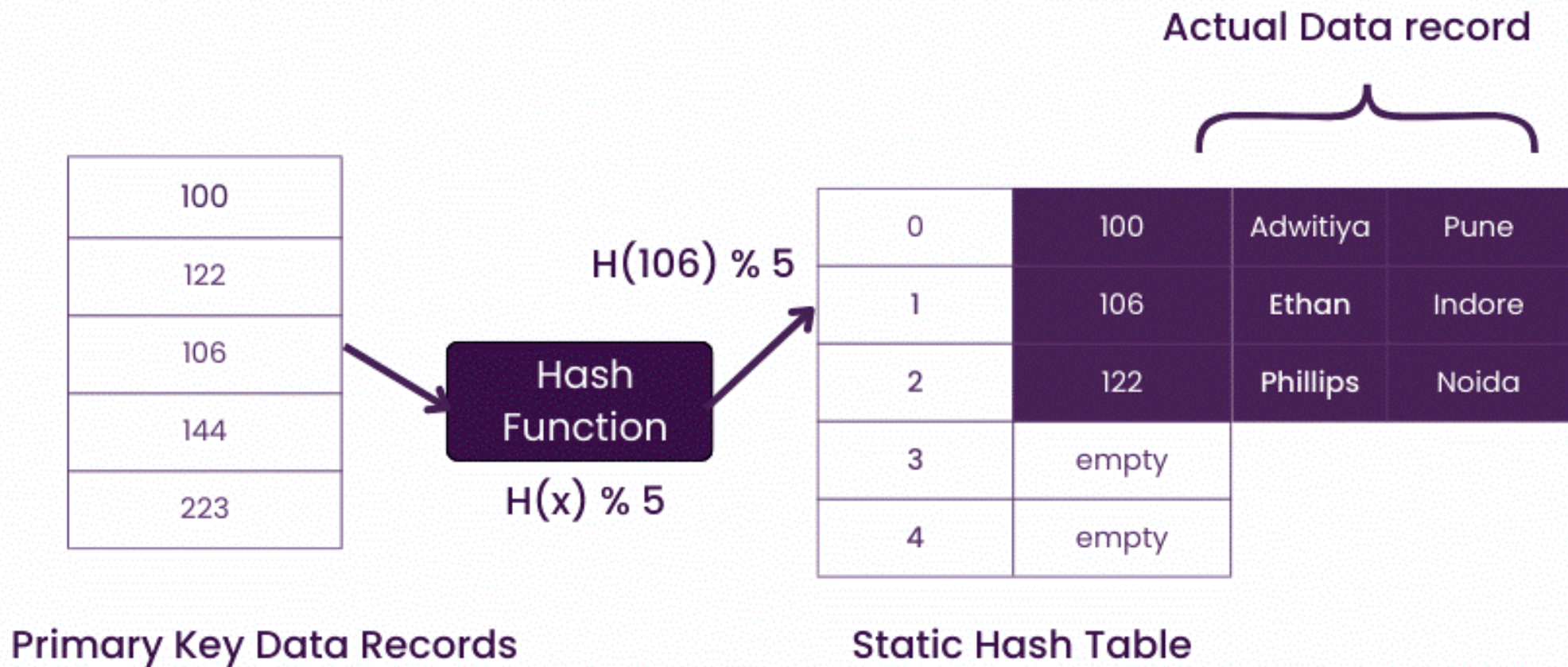
Hashing



静态哈希

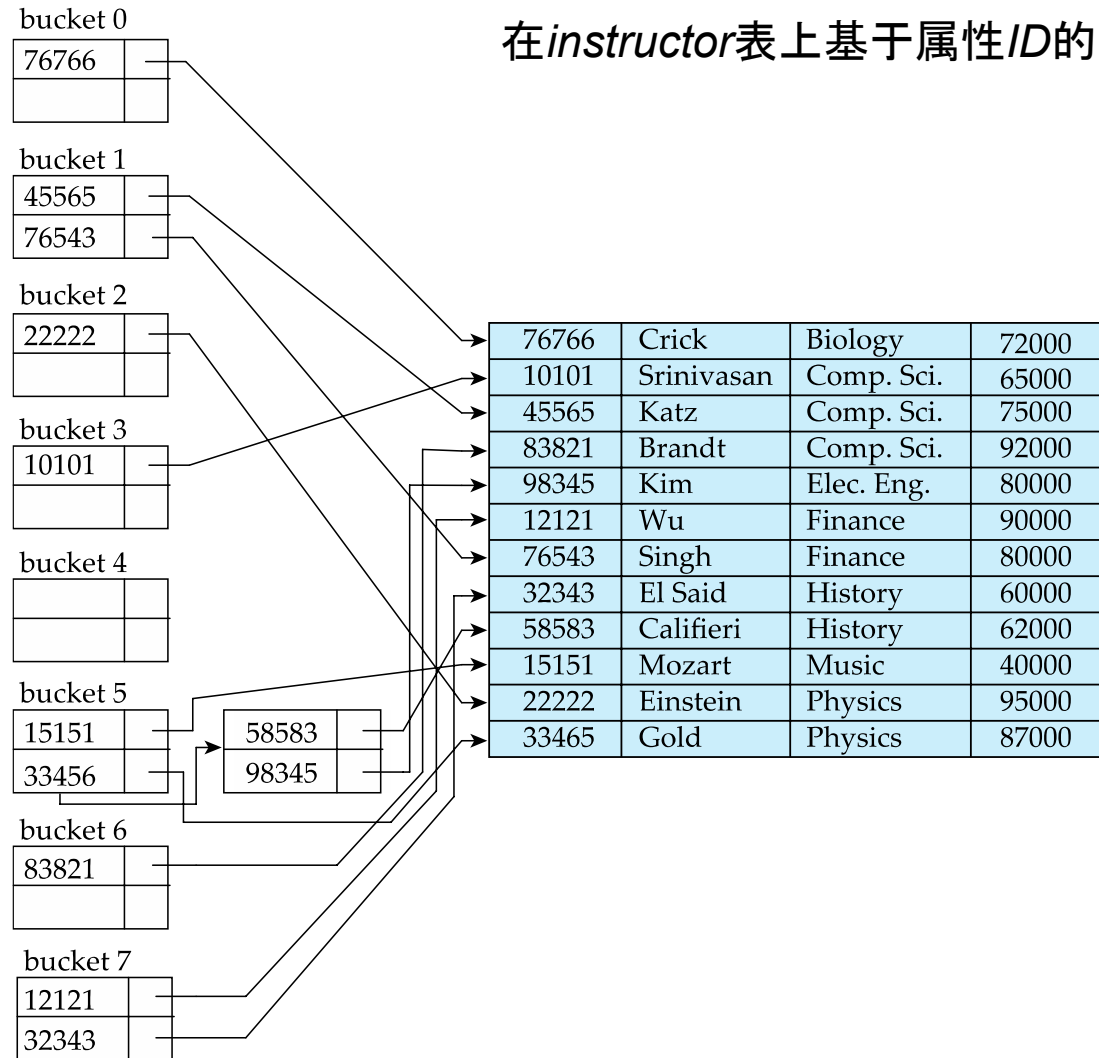
- 一个 **bucket** 是一个存储单元，包含一个或多个条目（在内存中 **bucket** 是指针数组，在硬盘中通常是一个磁盘区块 **block**）。
 - 我们可以通过一个哈希函数从 **搜索键** 获得一个条目的 **bucket**
- **哈希函数 h** 是从所有 **搜索键的集合 K** 到所有 **bucket 地址的集合 B** 的函数。
- 具有 **不同搜索键值** 的条目 **可能映射到同一个 bucket** 中；因此，必须按顺序搜索整个 **bucket** 才能找到条目。
- 在一个 **哈希索引** 中，**bucket** 存储指向记录的指针。
- 在一个 **哈希文件结构** 中，**bucket** 存储记录。

静态哈希



哈希索引示例

在 *instructor* 表上基于属性 *ID* 的哈希索引

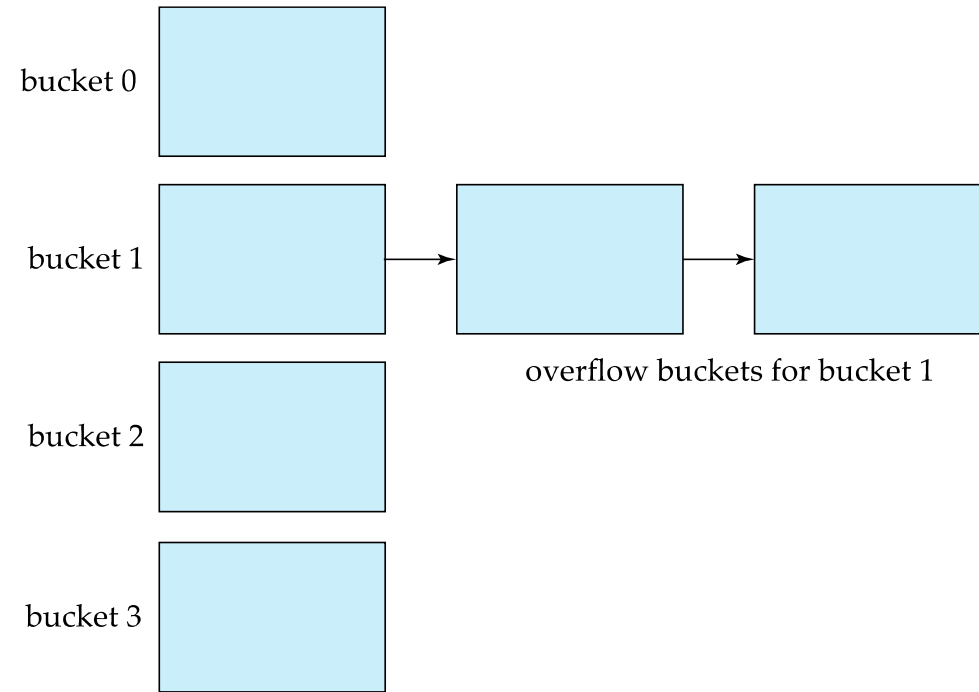


bucket 溢出处理

- bucket 溢出可能是由于以下原因造成的：
 - bucket 不足
 - 记录分布存在偏斜。这可能是由以下两个原因造成的：
 - 多条记录具有相同的搜索键值
 - 所选的哈希函数导致键值分布不均匀。
- 虽然可以降低 bucket 溢出的概率，但无法消除；这是通过使用 *overflow buckets* 来处理的。

bucket 溢出处理 (续)

- **overflow chaining** - 给定存储表的溢出表通过链表链接在一起。
- 上述方案称为 **封闭寻址**（**开放哈希**）。
 - 另一种方法是**开放寻址**（**封闭哈希**）不使用溢出存储表，因此不适合数据库应用程序。



哈希文件组织示例

使用 *dept_name* 作为键, 对 *instructor* 文件 进行哈希文件组织。

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

静态哈希的缺陷

- 在静态哈希中，函数 h 将搜索键值映射到一组固定的地址中。数据库会随着时间推移而增长或缩小。
 - 如果初始存储表的数量太少，而文件不断增长，则由于溢出过多，性能会下降。
 - 如果为预期增长预留空间，最初会浪费大量空间 (而且存储表会装不满)。
 - 如果数据库缩小，又会造成空间浪费。
- 一种解决方案：定期使用新的哈希函数对文件进行重新组织。
 - 成本高昂，扰乱正常的操作
- 更好的解决方案：允许动态修改存储表的数量。

动态哈希

- 定期重新哈希

- 例如, 如果哈希表中的条目数量变为哈希表大小的 1.5 倍,
 - 创建一个新的哈希表, 其大小是先前哈希表大小的 2 倍。
 - 将所有条目重新通过哈希函数写入新表

- 线性哈希

- 以增量方式进行重新哈希

- 可扩展哈希

- 专为基于磁盘的哈希而设计, 多个哈希值共享同一个存储表。
- 哈希表中的条目数翻倍, 但存储表数不翻倍

有序索引 与 哈希 的比较

- 定期重组的成本
- 插入和缺失的相对频率
- 为了优化平均访问时间而牺牲最坏情况下的访问时间是否可取？
- 预期查询类型：
 - 哈希算法通常更擅长检索具有指定键值的记录。
 - 如果范围查询很常见，则应优先选择有序索引。
- 实际操作中：
 - PostgreSQL 支持哈希索引，但由于性能不佳，不建议使用。
 - Oracle 支持静态哈希结构，但不支持哈希索引。
 - SQL Server 仅支持B⁺树。

多键访问

- 对于某些类型的查询，可以使用多个索引。

- 例子：

```
select ID
```

```
from instructor
```

```
where dept_name = "Finance" and salary = 80000;
```

- 使用单属性索引处理查询的可能策略：

1. 使用在 *dept_name* 上的索引查找系名为 Finance 的教员；测试 *salary* = 80000
2. 使用在 *salary* 上的索引查找薪资为 80000 美元的教员；测试 *dept_name* = "Finance"。
3. 使用 *dept_name* 的索引查找指向与 "Finance" 系的相关的所有记录的指针。类似地，使用 *salary* 的索引查找。取两组指针的交集。

多键索引

- **复合搜索键** 搜索键包含多个属性
 - 例如, (*dept_name*, *salary*)
- 字典序排序: $(a_1, a_2) < (b_1, b_2)$, 如果其中之一
 - $a_1 < b_1$, 或者
 - $a_1 = b_1$ 且 $a_2 < b_2$

多属性索引

假设我们有一个基于组合搜索键的索引
(*dept_name*, *salary*).

- 使用where子句

where *dept_name* = “Finance” and *salary* = 80000;

可以使用(*dept_name*, *salary*) 索引来获取仅满足这两个条件的记录。

- 使用单独的索引效率较低 - 我们可能会获取许多只满足其中一个条件的记录 (或指针) 。

- 也能高效处理

where *dept_name* = “Finance” and *salary* < 80000;

- 但无法有效处理

where *dept_name* < “Finance” and *balance* = 80000;

- 可能会检索到许多满足第一个条件但不满足第二个条件的记录

其他功能

- 覆盖索引

- 向索引添加额外属性，以便（某些）查询可以避免获取实际记录。
- 仅在叶子节点存储额外属性
 - 为什么？

- 对二级索引尤其有用

- 为什么？

索引的创建

- 例子

```
create index takes_pk on takes (ID, course_id, year, semester, sec_id);  
drop index takes_pk;
```

- 大多数数据库系统允许指定 索引类型 和 聚类。
- 所有数据库都会 **自动创建主键索引**。
 - 为什么?
- 一些数据库还会为 **外键属性** 创建索引。
 - 为什么这样的索引对这个查询有用:
 - $takes \bowtie \sigma_{name='Shankar'}(student)$
- 索引可以极大地 **加快查找速度**，但会增加更新的成本。
 - 多个数据库支持索引调优助手/向导，可根据查询和更新工作负载帮助选择索引。

SQL 中的索引定义

- 创建索引

`create index <index-name> on <relation-name>
(<attribute-list>)`

例如: `create index b-index on branch(branch_name)`

- 使用 **create unique index** 来间接指定和强制执行搜索键是**候选键**的条件。

- 如果唯一完整性约束受支持，则实际上不需要此功能。

- 删除索引

`drop index <index-name>`

- 大多数数据库系统允许**指定索引类型**和**聚类**。

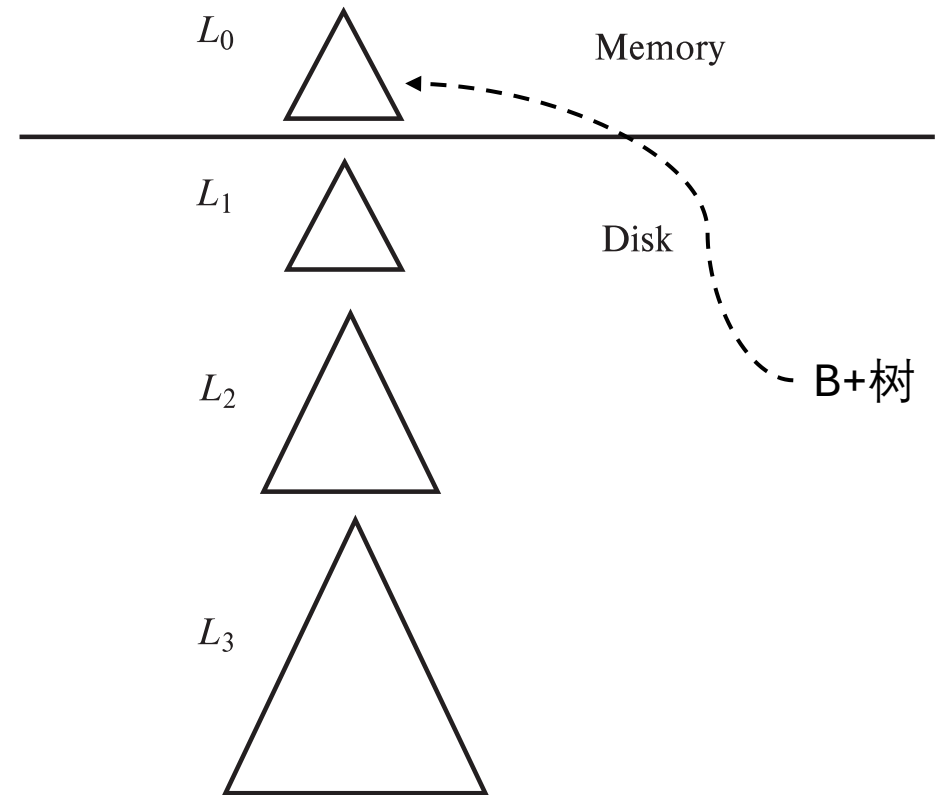
写入优化索引

- B⁺树在写入密集型工作负载下的性能可能较差。
 - 假设所有内部节点都位于内存中，则每个叶节点对应一个 I/O。
 - 使用磁盘时，每张磁盘每秒插入次数少于 100 次。
 - 使用闪存时，每次插入都会覆盖一页。
- 降低写入成本的两种方法
 - 日志结构合并树 (Log-structured merge tree)
 - 缓冲区树 (Buffer tree)

日志结构合并 (LSM) 树

(Log-structured merge tree)

- 目前只考虑插入/查询操作
- 首先将记录插入到**内存中的树(L_0 树)**中
- 当**内存树已满**时，记录会被移至**磁盘 (L_1 树)**。
 - **B⁺树**是通过将**现有的 L_1 树与 L_0 树中的记录合并**，采用自底向上构建方法构建的。
- 当 L_1 树超过某个阈值时，合并到 L_2 树中。
 - 如此循环往复，直至更多层级。
 - L_{i+1} 树的大小阈值是 L_i 树大小阈值的 k 倍



LSM树 (续)

• LSM方法的**优势**

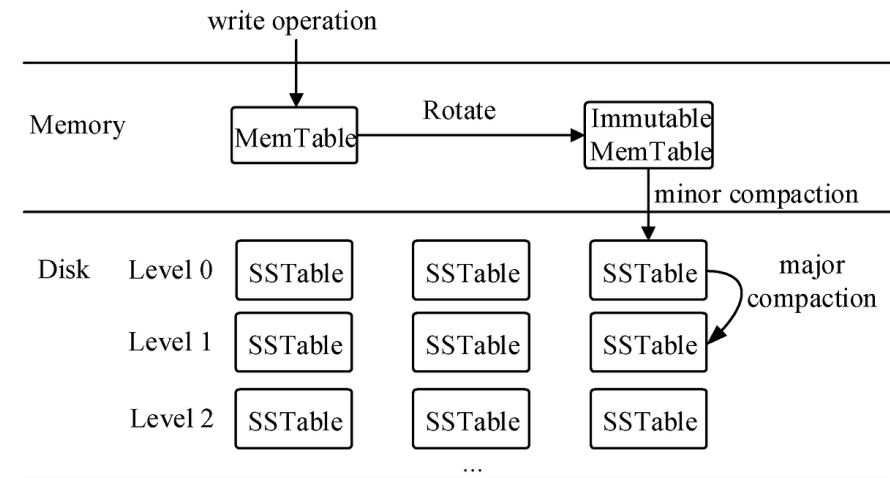
- 插入操作仅使用顺序 I/O 操作完成。
- 叶片是满的，从而避免空间浪费
- 与普通的 B+树相比，每次插入记录所需的 I/O 操作次数减少（在一定规模内）。

• LSM方法的**缺点**

- 查询需要**搜索多个树**。
- 每个层级的全部内容都被**多次复制**

• 分步合并 (Stepped-merge) 索引

- LSM树的变体，每一层都有多棵树。
- 与 LSM 树相比，降低了写入成本。
- 但查询成本更高。
 - 布隆过滤器 (Bloom filters) 可以避免在大多数树中进行查找。



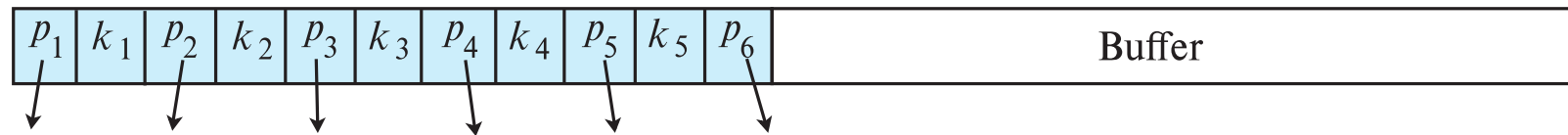
LSM树的另一种实现形式

LSM树 (续)

- 删除操作通过**添加特殊的“删除”条目**来处理。
 - **查找操作**会同时找到 **原始条目**和**删除条目**，并且必须仅返回那些没有匹配删除条目的条目。
 - 合并树时，如果发现删除条目与原始条目匹配，则两者都会被删除。
- 更新操作使用 **插入+删除** 功能处理。
- 引入LSM树曾是为了构建基于磁盘的索引。
 - 但对于**基于闪存的索引**来说，这有助于最大限度地减少擦除次数。
 - LSM树的分步合并变体被广泛应用于许多大数据存储系统中。
 - Google BigTable 、 Apache Cassandra、 MongoDB
 - 最近，SQLite4、 LevelDB 和MySQL的MyRocks存储引擎也采用了这种技术。

缓冲区树 Buffer Tree

Internal node



- LSM树的替代方案
- 核心思想：B⁺树的每个内部节点都有一个缓冲区来存储插入操作。
 - 当缓冲区满时，插入操作会被移至较低层级。
 - 使用较大的缓冲区时，每次都会将许多记录移动到较低级别。
 - 每个记录的 I/O 相应减少
- 好处
 - 查询开销更小
 - 可与任何树形索引结构一起使用
 - 用于 PostgreSQL 通用搜索树 (GiST) 索引
- 缺点：I/O 比 LSM 树更随机。

位图索引 (Bitmap Indices)

- **位图索引**是一种特殊类型的索引，专为高效查询**多个键**而设计。
- 关系中的记录假定从 0 开始按顺序编号。
 - 给定一个数字 n ，必须能够轻松地检索记录 n 。
 - 如果记录大小固定，则尤其容易操作。
- 适用于**取值相对较少**的属性。
 - 例如，性别、国家、州.....
 - 例如，收入水平（将收入分成少量等级，例如 0-9999、10000-19999、20000-50000、50000-无穷大）
- 位图本质上就是一个**二进制数组**。

位图索引 (续)

- 最简单的形式是，某个属性上的**位图索引**是为该属性的每个值都建立对应的一个位图。
 - 位图的**位数**与**记录的个数**相同
 - 如果编号为 i 的记录的属性 A 的值为 v_j ，则位图值 v_j 的第 i 位设置为 1。所有其他位都设置为 0。

• 例子：

record number	ID	gender	income_level
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1
3	15151	m	L4
4	58583	f	L3

Bitmaps for gender

m	10010
f	01101

Bitmaps for income_level

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

位图索引 (续)

- 位图索引对于查询多个属性非常有用。

- 对于单属性查询来说不太有用

- 查询通过位图操作来回答。

- 交集 (和 and)

- 联合 (或 or)

- 每个操作都会获取两个大小相同的位图，并对对应的位应用操作，从而得到结果位图。

- 例如, $100110 \text{ AND } 110011 = 100010$

- $100110 \text{ OR } 110011 = 110111$

- $\text{NOT } 100110 = 011001$

- 收入水平为 **L1** 的**男性**: $10010 \text{ AND } 10100 = 10000$

- 然后可以检索所需的元组。

- 计算匹配元组的数量速度更快。

```
SELECT *  
FROM instructor_info  
WHERE gender = 'f' AND income_level = 'L2';
```

01100

01000

位图索引 (续)

- 位图索引通常远小于关系的大小。
 - 例如，如果记录为 100 字节，则单个位图的空间是关系所用空间的 $1/800$ 。
 - 如果不同属性值的数量为 8，则位图仅占关系大小的 1%。

位图操作的高效实现

- 位图 被打包成 字 (words); 一个字的与运算 (一条基本的 CPU 指令) 一次计算 32 位或 64 位的与运算结果。
 - 只需 31,250 条指令 (32-bit 指令) 即可对 100 万位的映射进行与运算。
- 用一个小技巧可以快速数出 1 的个数:
 - 使用每个字节作为索引, 访问预先计算好的包含 256 个元素的数组, 每个元素存储二进制表示中 1 的个数。
 - 可以使用字节对来进一步加速, 但会增加内存成本。
 - 将检索到的数据加起来
- 对于具有大量匹配记录的值, 可以在 B⁺ 树的叶子节点级别使用位图代替元组 ID 列表。
 - 如果超过 1/64 的记录具有该值 (假设元组 ID 为 64 位), 则值得这样做。
 - 上述技术融合了位图索引和 B⁺ 树索引的优点。

空间和时间索引

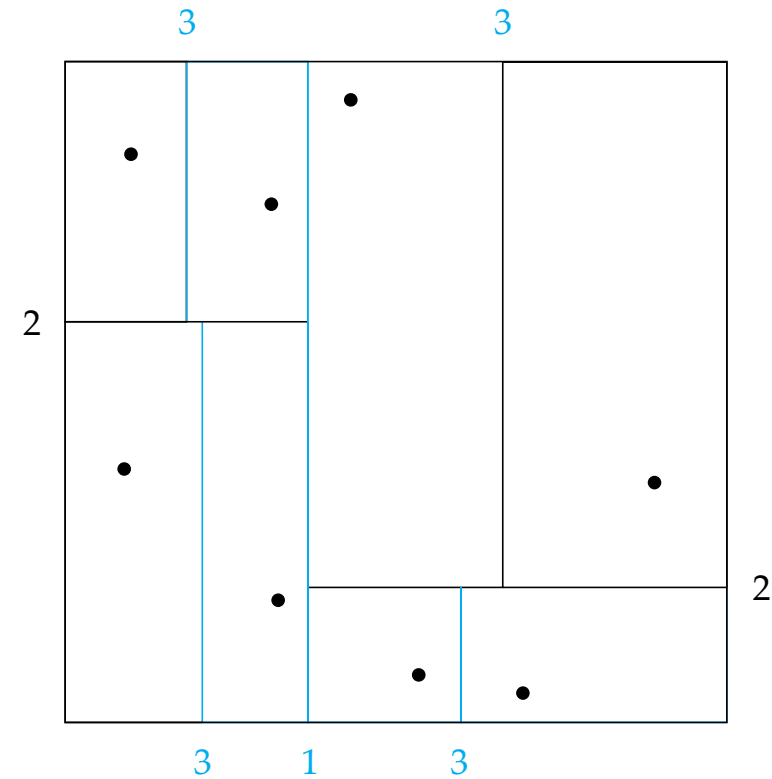
Spatial and Temporal Index

空间数据

- 数据库除了可以存储**栅格图像**外，还可以存储**线**、**面**等数据类型。
 - 关系数据库允许存储和检索空间信息
 - 查询可以使用空间条件（例如包含或重叠）。
 - 查询可以混合空间和非空间条件
- **最近邻查询**，给定一个点或一个对象，找到满足给定条件的最近对象。
- **范围查询**处理的是空间区域。例如，查询部分或全部位于指定区域内的对象。
- 计算区域交集或**并集的查询**。
- 以位置为连接属性，对两个空间关系进行**空间连接**。

空间数据索引

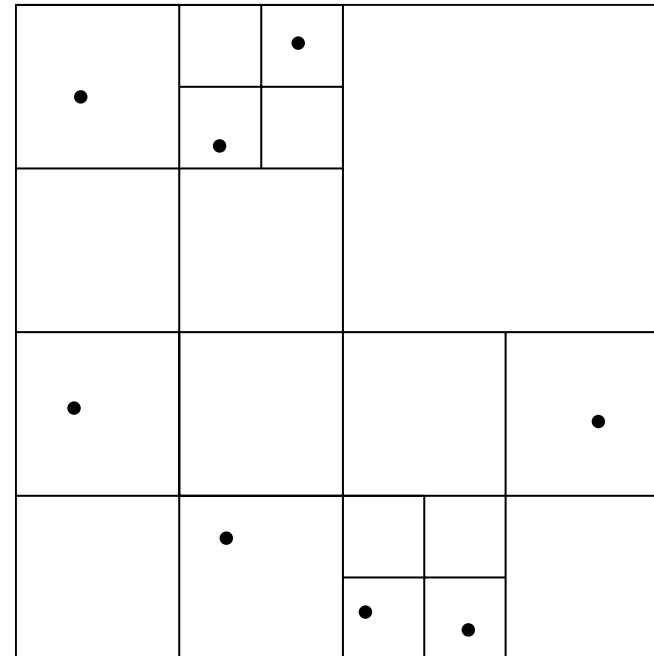
- **k - d 树 (k-dimensional tree)** - 用于多维索引的早期结构。
- k - d 树的每一层都将空间分割成两部分。
 - 在树的根级别选择一个维度进行分区。
 - 选择另一个维度对下一级节点进行分区，依此类推，循环遍历各个维度。
- 在每个节点中，存储在子树中的点大约有一半落在在一侧，另一半落在另一侧。
- 当节点上的点数少于给定数量时，分区停止。



- **kdB 树**扩展了 kd 树, 允许每个内部节点有多个子节点; 非常适合辅助存储。

用四叉树 (Quadrees) 划分空间

- 四叉树的每个节点都与空间中的一个矩形区域相关联；顶节点与整个目标空间相关联。
- 每个非叶节点将其区域划分为四个大小相等的象限。
 - 相应地，每个这样的节点都有四个子节点，分别对应于四个象限，依此类推。
- 叶节点的点数介于零个和某个固定的最大点数之间（例如，设置为 1）。

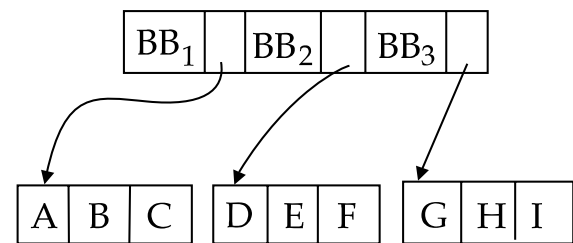
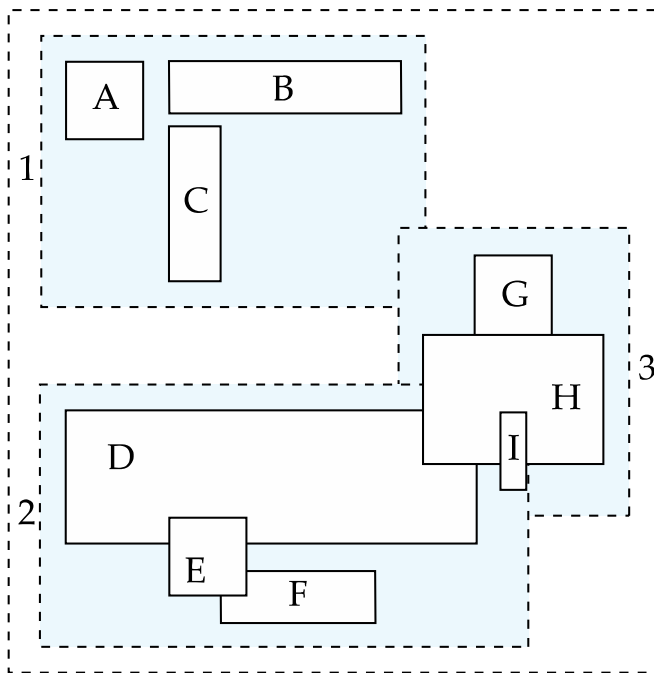


R树

- **R树** 是 **B⁺树** 的 **N 维扩展**，可用于索引**矩形 (rectangles)** 和其他多边形的集合。
- **B⁺树** 和 **R 树** 等变体。
- 基本思想：将与每个**B⁺树**节点关联的一维区间的概念推广到**N 维区间**，即 **N 维矩形**。
- 仅考虑二维情况（ $N=2$ ）
 - 对于 $N > 2$ 的情况，推广是直接的，尽管 **R 树** 仅在 **N** 相对较小时效果良好。
- 节点的**边界框**是一个最小尺寸的矩形，它包含与该节点关联的所有矩形/多边形。
 - 允许节点的子节点的边界框重叠。

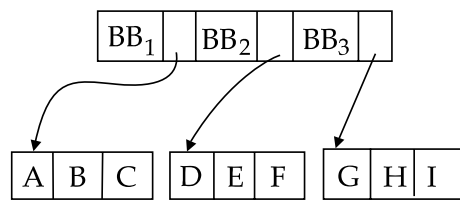
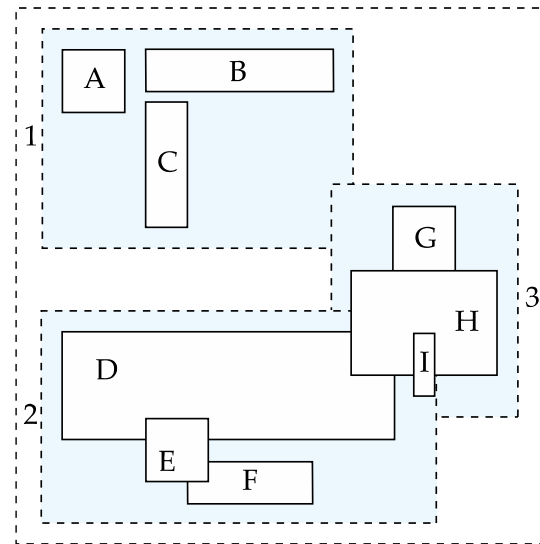
R 树示例

- 一组矩形（实线, A, B, C）和矩形的 R 树节点的边界框 **Bounding Box**（虚线, BB_1 , BB_2 , BB_3 ）。
- R树如图所示（右图）。



在 R 树中搜索

- 要查找与给定查询点/区域相交的数据项，从**根节点开始**执行以下操作：
 - 如果该节点是叶节点，则输出键与给定查询点/区域相交的数据项。
 - 否则，对于当前节点的每个子节点（其边界框与查询点/区域相交），递归地搜索该子节点。
- 最坏情况下效率可能很低，因为可能需要搜索多条路径，但在实践中效果尚可接受。



时间数据索引

- **时间数据**是指具有特定时间段（区间）的数据。
 - **course** 关系的时序版本

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>start</i>	<i>end</i>
BIO-101	Intro. to Biology	Biology	4	1985-01-01	9999-12-31
CS-201	Intro. to C	Comp. Sci.	4	1985-01-01	1999-01-01
CS-201	Intro. to Java	Comp. Sci.	4	1999-01-01	2010-01-01
CS-201	Intro. to Python	Comp. Sci.	4	2010-01-01	9999-12-31

- 时间间隔包含开始时间和结束时间。
 - 如果元组当前有效但其有效结束时间未知，则将结束时间设置为无穷大（或大日期，例如 **9999-12-31**）。
- 查询可以请求在某个**时间点**或某个**时间段**内所有有效的元组。
 - 对有效时间段进行索引可以加快这项任务。

时间序列数据索引 (续)

- 在属性 a 上创建时间索引：
 - 使用空间索引，例如 R 树，以属性 a 为一个维度，时间为另一个维度。
 - 有效时间在时间维度上构成一个区间。
 - 当前有效的元组会导致问题，因为其值可能是无限大或非常大。
 - 解决方案：将所有当前元组（结束时间为无穷大）存储在一个单独的索引中，索引基于 $(a, \text{start-time})$ 。
 - 在当前元组索引中时间点 t 有效的元组，在 $(a, 0)$ 到 (a, t) 的范围内搜索元组。

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>start</i>	<i>end</i>
BIO-101	Intro. to Biology	Biology	4	1985-01-01	9999-12-31
CS-201	Intro. to C	Comp. Sci.	4	1985-01-01	1999-01-01
CS-201	Intro. to Java	Comp. Sci.	4	1999-01-01	2010-01-01
CS-201	Intro. to Python	Comp. Sci.	4	2010-01-01	9999-12-31

- 主键上的时间索引有助于强制执行时间主键约束。

Database Systems

数据库系统

Instructor: Yiwen Shen (沈轶闻), Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr